# Lecture. Matplotlib Package. Pandas Data Visualization

Rostov-on-Don
2026

# Contents


**1 Introduction to the pyplot Interface** **3**

**2 Setting Styles of the plot Function** **4**

**3 Plotting with Categorical Variables** **7**

**4 Controlling Line Properties** **8**
4.1 List of Line2D Properties . . . . . 8

**5 Working with Multiple Figures and Axes** **9**

**6 Working with Text** **11**
6.1 Using Mathematical Expressions in Text . . . . . 12
6.2 Annotations . . . . . 12

**7 Logarithmic and Nonlinear Axes** **13**

**8 Pandas Data Visualization** **15**
8.1 Basics of Plotting: the plot Command . . . . . 15
8.2 Other Plots . . . . . 18

**9 Bar Plots** **19**

**10 Histograms** **21**

**11 Box Plots** **25**

**12 Area Plot** **30**

**13 Scatter Plot** **32**

**14 Hexagonal Bin Plot** **34**

**15 Pie Plot** **36**

**16 Plotting with Missing Data** **37**

**17 Plotting Tools** **38**
17.1 Scatter Matrix Plot . . . . . 38
17.2 KDE Plot . . . . . 38

**18 Introduction to seaborn** **39**
18.1 High-level API for Statistical Graphics . . . . . 41
18.2 Statistical Estimation . . . . . 42
18.3 Distributional Representations . . . . . 42
18.4 Plots for Categorical Data . . . . . 43
18.5 Multivariate Representations for Complex Datasets . . . . . 45
18.6 Confident Defaults and Flexible Customization . . . . . 48

18.7 Relationship with matplotlib . . . . . 50
18.8 Next Steps . . . . . 50

**References** **50**

# 1 Introduction to the pyplot Interface

https://matplotlib.org/stable/tutorials/introductory/pyplot.html

`matplotlib.pyplot` is a collection of matplotlib functions with graphical commands in the MATLAB style. Each pyplot function changes some parameters of the figure: for example, it creates a figure (a window for drawing), creates a drawing area on the figure, draws some lines in the plotting area, adds labels, etc.

In `matplotlib.pyplot`, various states are preserved across function calls, so it keeps track of things like the current figure and plotting area.

## Note

The pyplot API is generally less flexible than the object-oriented API. Most of the function calls you see here can also be called as methods from an `Axes` object (with the same or similar names).

Outputting a plot using pyplot:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
plt.show()
```

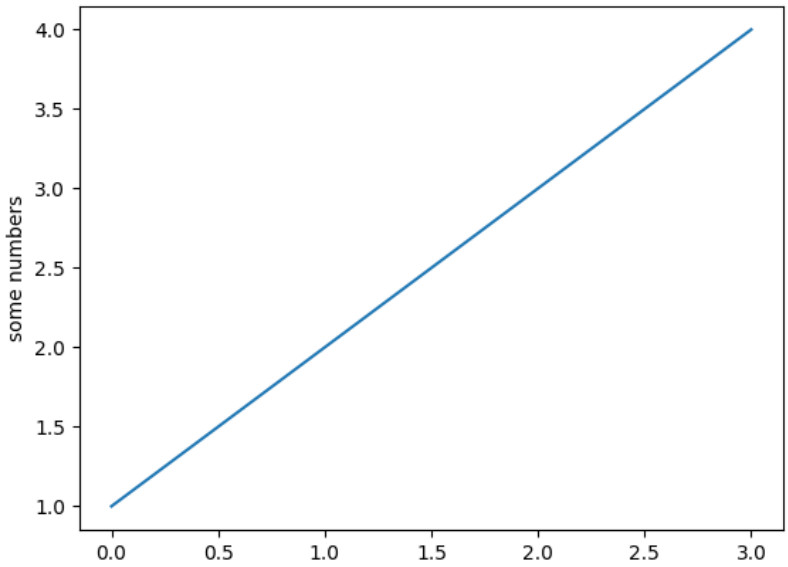



Figure 1: Line plot with ylabel “some numbers”

You may be wondering why the x-axis ranges from 0 to 3 and the y-axis from 1 to 4. If you pass a single list to plot, matplotlib assumes it is the sequence of y values, and automatically generates the x values as `[0, 1, 2, 3]`.

`plot` is a function with a large number of possible parameters. For example, to plot the graph of `(x, y)`, you can write:

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

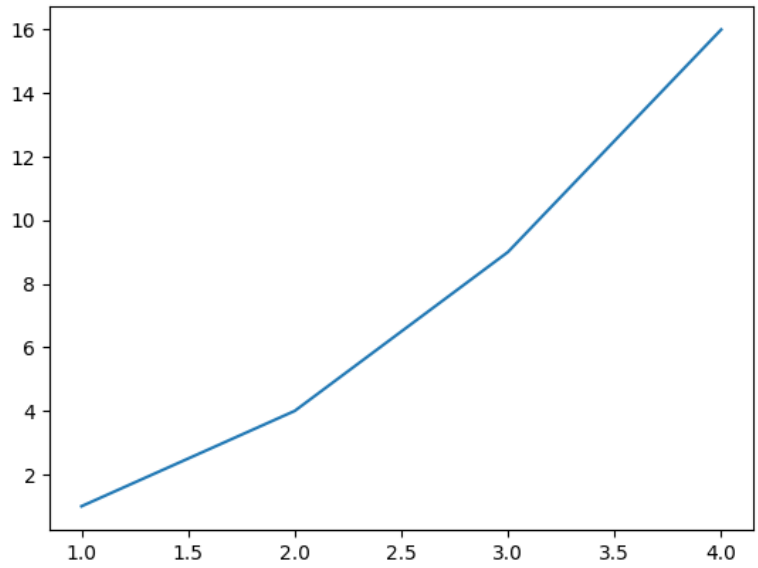



Figure 2: Line plot of $x^2$

```
Out: [<matplotlib.lines.Line2D object at 0x7f1c6bee4940>]
```

## 2 Setting Styles of the plot Function

For each pair of arguments $x$, $y$ there is a third optional argument, a format string, that specifies the color and line type of the plot. The letters and symbols of the format string are the same as in MATLAB. The default format string is `"b-"`, which means a solid blue line. For example, to plot red circles, you would write `'ro'` (r – red; o – circle):

```
1 plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
2 plt.axis([0, 6, 0, 20])
3 plt.show()
```

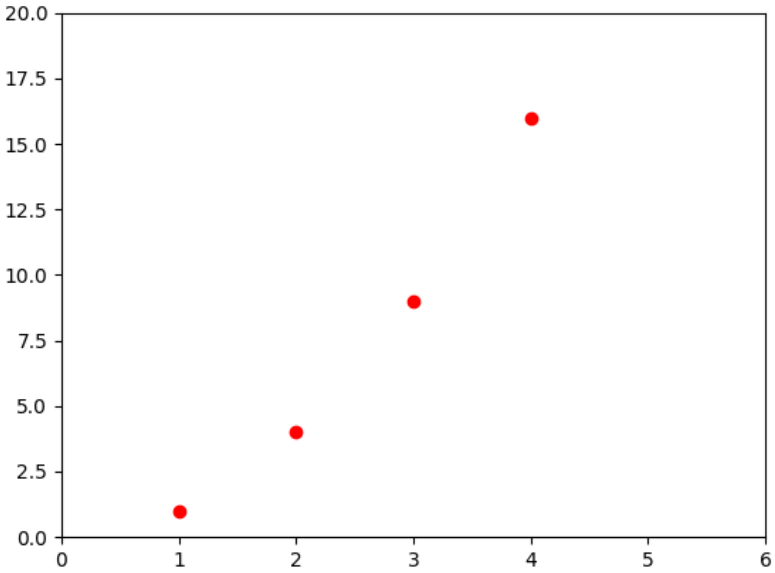



Figure 3: Red circles scatter plot

For the full list of line styles and format strings, see the plot documentation. The `axis` function in the example above takes a list `[xmin, xmax, ymin, ymax]` and specifies the dimensions along the axes.

If matplotlib were limited to working with lists, it would be completely useless for numerical processing. As a rule, you will use numpy arrays. In fact, all data is internally converted to numpy arrays. The example below shows how to plot several graphs with different styles in a single call to the `plot` function using arrays.

```
import numpy as np

# evenly sampled time at 200ms intervals
t = np.arange(0., 5., 0.2)

# red dashes, blue squares and green triangles
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```

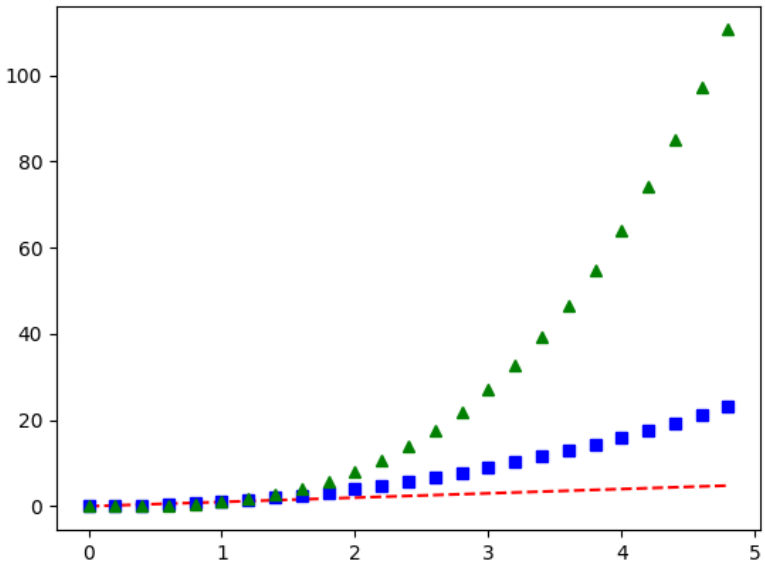



Figure 4: Multiple line styles plot

Some commands that have a `data` parameter allow you to pass data as a dictionary. For example, the commands `numpy.recarray` or `pandas.DataFrame`.

```
data = {'a' : np.arange(50),
        'c' : np.random.randint(0, 50, 50),
        'd' : np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

plt.scatter('a', 'b', c='c', s='d', data=data)
plt.xlabel('entry a')
plt.ylabel('entry b')
plt.show()
```

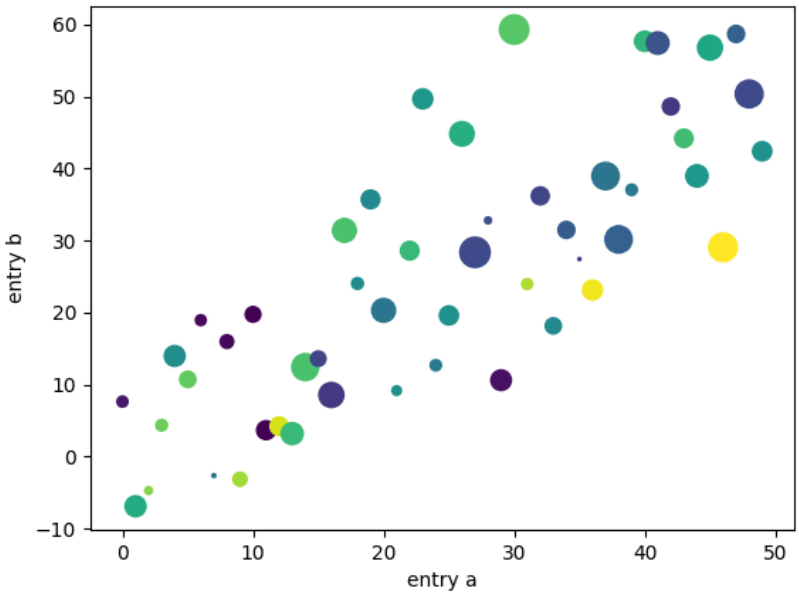



Figure 5: Scatter plot with color and size mapping

# 3 Plotting with Categorical Variables

Matplotlib allows you to pass categorical variables directly to many plotting functions. For example:

```
names = ['group_a', 'group_b', 'group_c']  # x-axis - categories
values = [1, 10, 100]

plt.figure(figsize=(9, 3))  # window size

plt.subplot(131)  # set 3 windows and select the 1st one
plt.bar(names, values)
plt.subplot(132)  # 2nd window
plt.scatter(names, values)
plt.subplot(133)  # 3rd window
plt.plot(names, values)
plt.suptitle('Categorical Plotting')
plt.show()
```

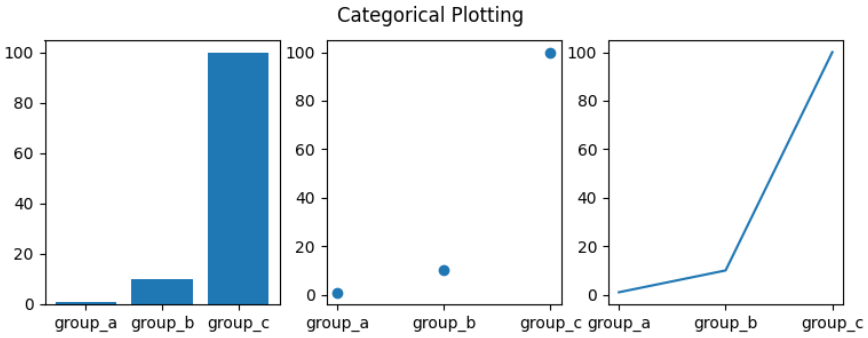



Figure 6: Categorical plotting: bar, scatter, line

# 4 Controlling Line Properties

Lines have many attributes: `linewidth`, dash style, `antialiased`, etc.; see `matplotlib.lines.Line2D`. There are several ways to set line properties.

- Using keyword arguments:

```
plt.plot(x, y, linewidth=2.0)
```

- Using set-methods of `Line2D` line objects. The `plot` command returns a list of such objects; for example, `line1, line2 = plot(x1, y1, x2, y2)`.

```
line, = plt.plot(x, y, '-')
line.set_antialiased(False)  # turn off antialiasing
```

- Using the `setp` method. The example below uses a MATLAB-style function to set multiple properties. You can use either assignments (parameter = value) or a set of parameter-value pairs.

```
lines = plt.plot(x1, y1, x2, y2)
# use keyword args
plt.setp(lines, color='r', linewidth=2.0)
# or MATLAB style string value pairs
plt.setp(lines, 'color', 'r', 'linewidth', 2.0)
```

## 4.1 List of Line2D Properties

| Property | Value Type |
|---|---|
| alpha | float |
| animated | [True \| False] |
| antialiased or aa | [True \| False] |
| clip_box | a matplotlib.transform.Bbox instance |
| clip_on | [True \| False] |
| clip_path | a Path instance and a Transform instance, a Patch |
| color or c | any matplotlib color |

| Property | Value Type |
|---|---|
| contains | the hit testing function |
| dash_capstyle | ['butt' \| 'round' \| 'projecting'] |
| dash_joinstyle | ['miter' \| 'round' \| 'bevel'] |
| dashes | sequence of on/off ink in points |
| data | (np.array xdata, np.array ydata) |
| figure | a matplotlib.figure.Figure instance |
| label | any string |
| linestyle or ls | ['-' \| '–' \| '-.' \| ':' \| 'steps' \| ...] |
| linewidth or lw | float value in points |
| marker | ['+' \| ',' \| '.' \| '1' \| '2' \| '3' \| '4'] |
| markeredgecolor or mec | any matplotlib color |
| markeredgewidth or mew | float value in points |
| markerfacecolor or mfc | any matplotlib color |
| markersize or ms | float |
| markevery | [None \| integer \| (startind, stride)] |
| picker | used in interactive line selection |
| pickradius | the line pick selection radius |
| solid_capstyle | ['butt' \| 'round' \| 'projecting'] |
| solid_joinstyle | ['miter' \| 'round' \| 'bevel'] |
| transform | a matplotlib.transforms.Transform instance |
| visible | [True \| False] |
| xdata | np.array |
| ydata | np.array |
| zorder | any number |

To get a list of all properties, call the `setp` function with only the line object, without other parameters:

```
In [69]: lines = plt.plot([1, 2, 3])
In [70]: plt.setp(lines)
  alpha: float
  animated: [True | False]
  antialiased or aa: [True | False]
  ...snip
```

# 5 Working with Multiple Figures and Axes

In MATLAB and pyplot, there is a concept of the current figure (window) and current axes (a single window can have multiple axes). All operations apply to the current window and axes. The `gca` function returns a reference to the current axes (a `matplotlib.axes.Axes` object), and the `gcf` function returns a reference to the current window (a `matplotlib.figure.Figure` object). Usually you don't need to worry about this. Below is a script with two axes in one window.

```
1 def f(t):
2     return np.exp(-t) * np.cos(2*np.pi*t)
3
```

```
t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)

plt.figure()
plt.subplot(211)  # or: plt.subplot(2, 1, 1)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')

plt.subplot(212)
plt.plot(t2, np.cos(2*np.pi*t2), 'r--')
plt.show()
```

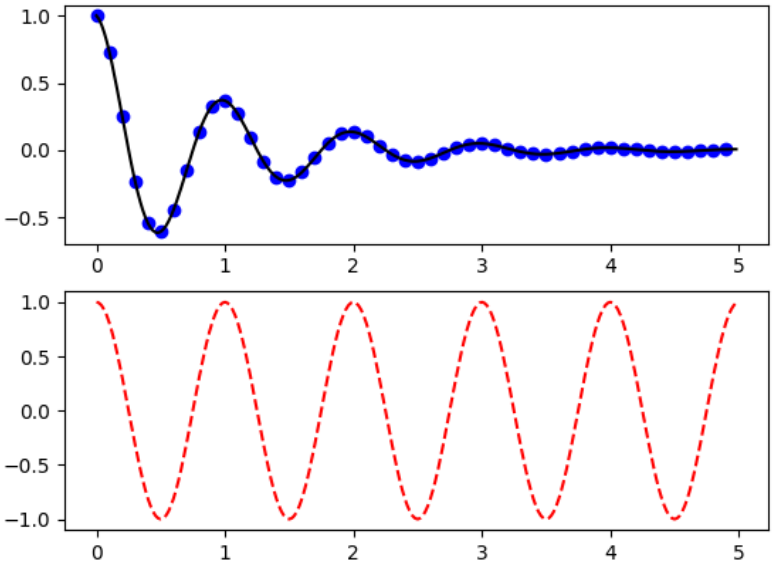



Figure 7: Two subplots: damped oscillation and cosine

Calling the `figure` command is optional because a figure will be created automatically if there is no explicit call. Axes will also be created (equivalent to an explicit call to `subplot()`) if there is no explicit call.

The `subplot` command is passed the parameters `numrows`, `numcols`, `plot_number`, where `plot_number` ranges from 1 to `numrows*numcols`. Commas between parameters can be omitted if `numrows*numcols < 10`. The call `subplot(211)` is identical to `subplot(2, 1, 1)`.

You can create an arbitrary number of subplots and axes. If you want to place axes manually, i.e., not on a rectangular grid, use the `axes` command, which allows you to specify the location as `axes([left, bottom, width, height])`, where all values are given as fractions (from 0 to 1) – relative coordinates of the window placement. See the Axes Demo for examples of manual axes placement and the Basic Subplot Demo for examples with a large number of subplots.

You can create multiple figures by calling them with different numbers. Each figure can contain multiple axes and subplots:

```
import matplotlib.pyplot as plt
plt.figure(1)                # the first figure
```

```
plt.subplot(211)              # the first subplot in the first
    figure
plt.plot([1, 2, 3])
plt.subplot(212)              # the second subplot in the first
    figure
plt.plot([4, 5, 6])

plt.figure(2)                 # a second figure
plt.plot([4, 5, 6])           # creates a subplot() by default

plt.figure(1)                 # figure 1 current; subplot(212) still
    current
plt.subplot(211)              # make subplot(211) in figure1 current
plt.title('Easy as 1, 2, 3') # subplot 211 title
```

You can clear the current figure with the `clf` command, and the current axes with `cla`. Using current values (in particular, the current image, figure, and axes) is just a thin stateful wrapper around the object-oriented API. If this is inconvenient, you can set them yourself instead (see the Artist tutorial).

If you are making many figures, you need to know one more thing: the memory required for a figure is not completely released until the figure is explicitly closed with `close`. Deleting all references to the figure and/or using the window manager to destroy the window in which the figure appears on the screen is not enough, since pyplot maintains internal references until close is called.

# 6 Working with Text

The `text` command can be used to add text at an arbitrary location in the figure, while `xlabel`, `ylabel`, and `title` are used to add text at specific locations (for a more detailed example, see the section Text in Matplotlib Plots).

```
mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50, density=1, facecolor='g', alpha
    =0.75)

plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)
plt.show()
```

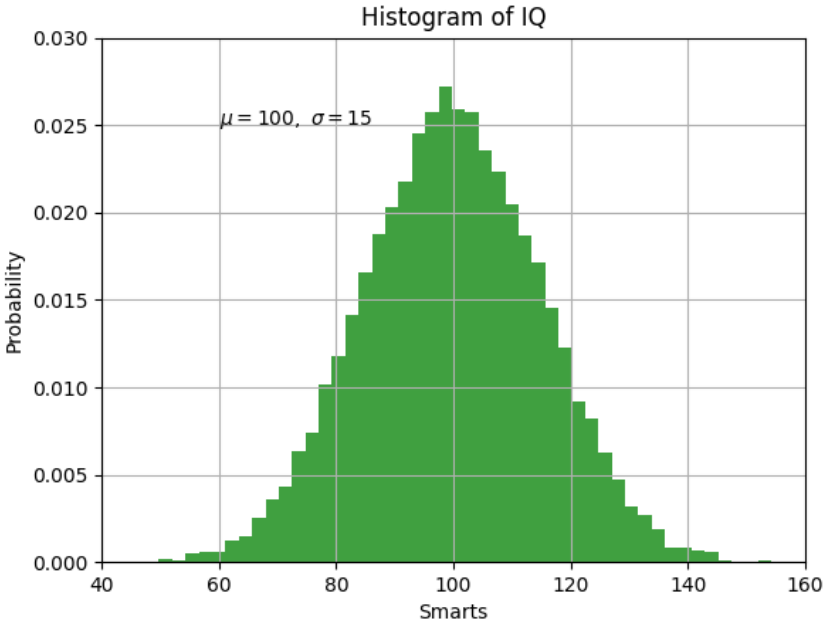



Figure 8: Histogram of IQ with text annotation

All text functions return `matplotlib.text.Text` objects. Just as with line properties, you can set properties using keyword arguments inside the text function or by using the `setp` function:

```
t = plt.xlabel('my data', fontsize=14, color='red')
```

These properties are described in more detail in Text properties and layout.

## 6.1 Using Mathematical Expressions in Text

matplotlib supports TeX. For example, to write the expression $\sigma_i = 15$ in the title, you write the TeX expression inside dollar signs:

```
plt.title(r'$\sigma_i=15$')
```

The `r` preceding the string is important – it means this is a raw string and backslashes should not be treated as Python escapes. matplotlib has a built-in TeX expression parser and layout engine, and ships its own math fonts – see the section Writing mathematical expressions for details. Thus, you can use mathematical text across platforms without installing TeX. For those who have LaTeX and dvipng installed, you can also use LaTeX to format text and include the output directly in displayed figures or saved postscript – see the section Text rendering With LaTeX.

## 6.2 Annotations

The `text` function places text at an arbitrary location. Usually text is used to label some part of the figure, and the `annotate` method provides auxiliary functionality. In `annotate`, two points are specified: the location of the annotated object `xy` and the location of the text `xytext`. Both of these arguments are given as tuples of the form `(x, y)`.

```
ax = plt.subplot()

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2*np.pi*t)
line, = plt.plot(t, s, lw=2)

plt.annotate('local max', xy=(2, 1), xytext=(3, 1.5),
             arrowprops=dict(facecolor='black', shrink=0.05))
plt.ylim(-2, 2)
plt.show()
```

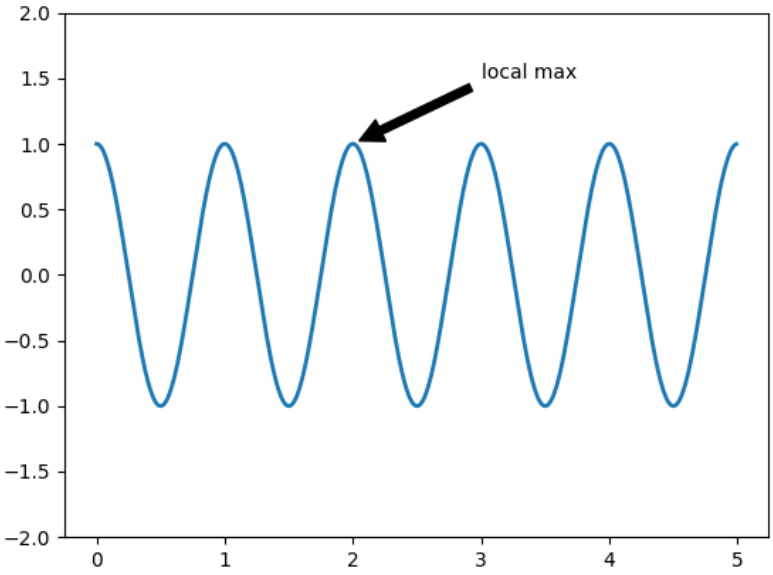



Figure 9: Cosine plot with annotation arrow

For more detailed information, see Basic annotation and Advanced Annotations. Many examples can be found in Annotating Plots.

# 7 Logarithmic and Nonlinear Axes

`matplotlib.pyplot` supports not only linear axes, but also logarithmic and logit axes. They are usually used when data varies over several orders of magnitude. Changing axes is simple: `plt.xscale('log')`

Below is an example of four plots with the same data and different scales.

```
# Fixing random state for reproducibility
np.random.seed(19680801)

# make up some data in the open interval (0, 1)
y = np.random.normal(loc=0.5, scale=0.4, size=1000)
y = y[(y > 0) & (y < 1)]
```

```
y.sort()
x = np.arange(len(y))

# plot with various axes scales
plt.figure()

# linear
plt.subplot(221)
plt.plot(x, y)
plt.yscale('linear')
plt.title('linear')
plt.grid(True)

# log
plt.subplot(222)
plt.plot(x, y)
plt.yscale('log')
plt.title('log')
plt.grid(True)

# symmetric log
plt.subplot(223)
plt.plot(x, y - y.mean())
plt.yscale('symlog', linthresh=0.01)
plt.title('symlog')
plt.grid(True)

# logit
plt.subplot(224)
plt.plot(x, y)
plt.yscale('logit')
plt.title('logit')
plt.grid(True)

# Adjust the subplot layout, because the logit one may take more
    space
# than usual, due to y-tick labels like "1 - 10^{-3}"
plt.subplots_adjust(top=0.92, bottom=0.08, left=0.10, right=0.95,
                    hspace=0.25, wspace=0.35)
```

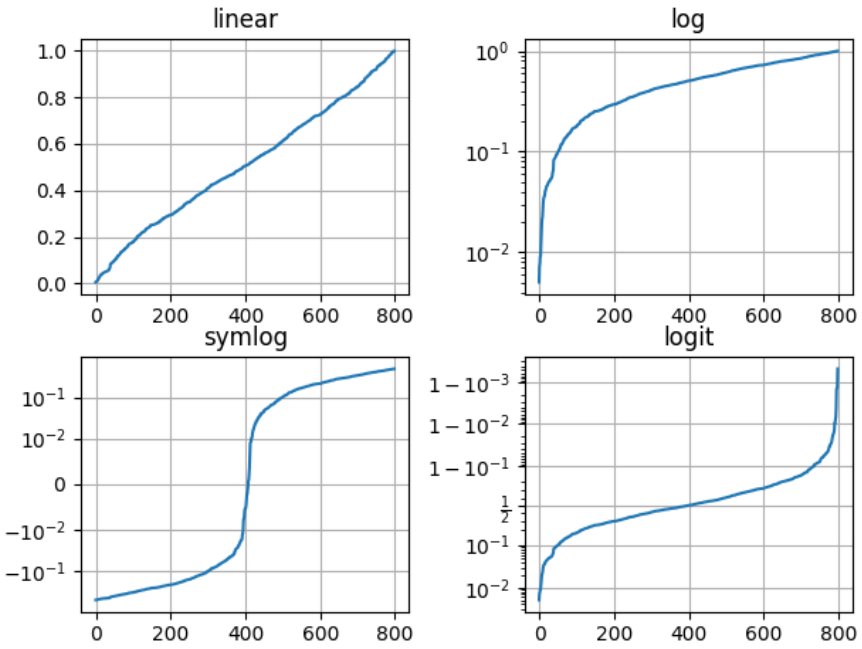



Figure 10: Four plots with different axes scales

You can add your own scale, see the Developer's guide for creating scales and transformations for details.

# 8 Pandas Data Visualization

https://pandas.pydata.org/pandas-docs/stable/user_guide/visualization.html

We use the standard convention for importing the matplotlib API:

```
In [1]: import matplotlib.pyplot as plt
        import numpy as np
        import pandas as pd
In [2]: plt.close("all")
```

We demonstrate the basic capabilities of pandas for creating plots. See the ecosystem section for an overview of additional visualization libraries that go beyond the basics described here.

**Note.** In all calls to `np.random`, the generator is initialized with the number 123456.

## 8.1 Basics of Plotting: the plot Command

We will demonstrate the basics; for more detailed information, see the cookbook.

The `plot` command on Series and DataFrame objects is simply a wrapper around the `plt.plot()` command:

```
In [3]: ts = pd.Series(np.random.randn(1000),
                       index=pd.date_range("1/1/2000", periods
   =1000))
In [4]: ts = ts.cumsum()
In [5]: ts.plot();
```

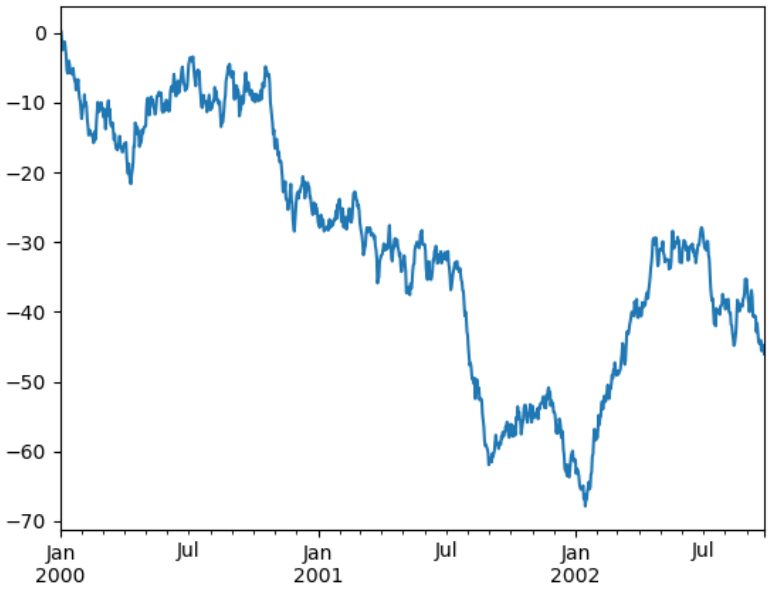



Figure 11: Time series line plot

If the index contains dates, it calls `gcf().autofmt_xdate()` to try to format the x-axis correctly, as shown above.

In a DataFrame, the `plot()` function is convenient for plotting all columns with labels:

```
In [6]: df = pd.DataFrame(np.random.randn(1000, 4), index=ts.index,
                          columns=list("ABCD"))
In [7]: df = df.cumsum()
In [8]: plt.figure();
In [9]: df.plot();
```

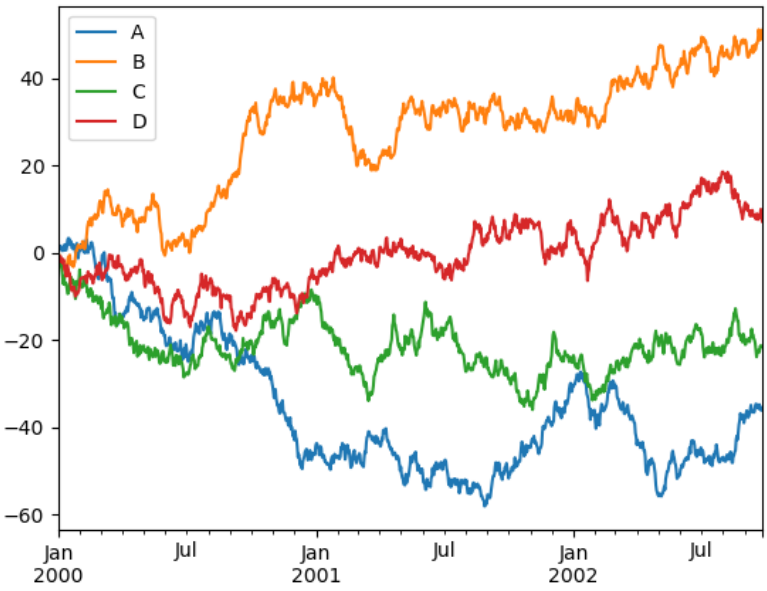



Figure 12: DataFrame line plot with legend

You can plot one column against another using the keyword arguments `x` and `y` in `plot()`:

```
In [10]: df3 = pd.DataFrame(np.random.randn(1000, 2), columns=['B',
    'C']).cumsum()
In [11]: df3["A"] = pd.Series(list(range(len(df))))
In [12]: df3.plot(x="A", y="B");
```

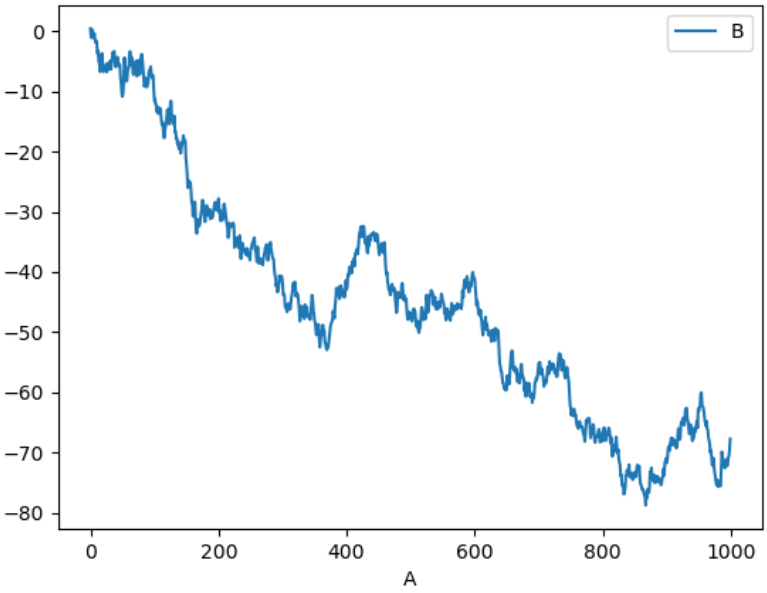



Figure 13: Line plot of column B vs A

**Note.** For additional formatting and style parameters, see the formatting section below.

## 8.2 Other Plots

Plotting methods allow you to use several plotting styles besides the default line plot. These methods can be provided as a keyword argument `kind` to `plot()` and include:

- `'bar'` or `'barh'` for rectangles
- `'hist'` for histogram
- `'box'` for boxplot
- `'kde'` or `'density'` for density plots
- `'area'` for area plot
- `'scatter'` for scatter plot
- `'hexbin'` for hexagonal bin plots
- `'pie'` for pie plots

For example, a histogram (bar type) can be created as follows:

```
In [13]: plt.figure();
In [14]: df.iloc[5].plot(kind="bar");
```

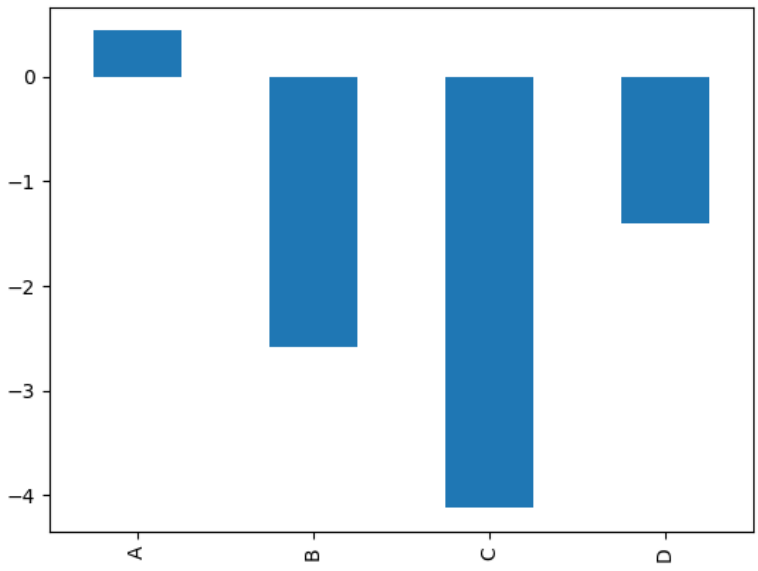



Figure 14: Bar plot of one row

You can also create other plots using `DataFrame.plot.<kind>` instead of specifying the `kind` argument.

```
In [15]: df = pd.DataFrame()
In [16]: df.plot.<TAB>  # noqa: E225, E999
df.plot.area      df.plot.barh      df.plot.density   df.plot.hist
df.plot.line      df.plot.scatter   df.plot.bar       df.plot.box
df.plot.hexbin    df.plot.kde       df.plot.pie
```

# 9 Bar Plots

For labeled data that is not time series, you can create a bar plot:

```
In [17]: plt.figure();
In [18]: df.iloc[5].plot.bar();
In [19]: plt.axhline(0, color="k");
```

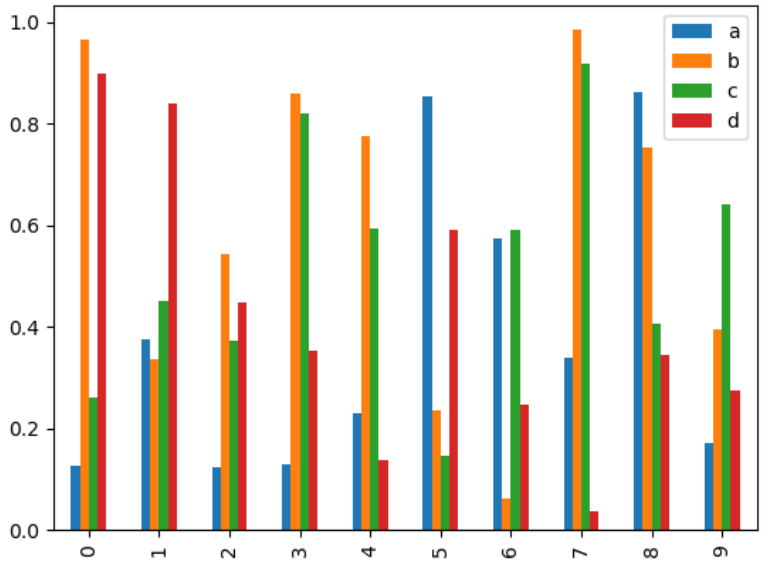



Figure 15: Bar plot with horizontal line at 0

Calling the DataFrame's `plot.bar()` method creates multiple bar plots:

```
In [20]: df2 = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b
    ', 'c', 'd'])
In [21]: df2.plot.bar();
```

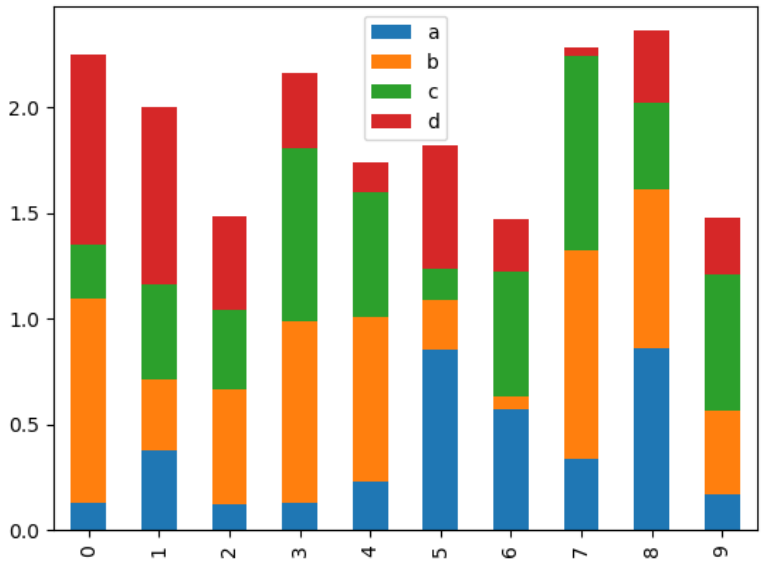



Figure 16: Grouped bar plot

To create a stacked bar chart, set `stacked=True`:

```
In [22]: df2.plot.bar(stacked=True);
```

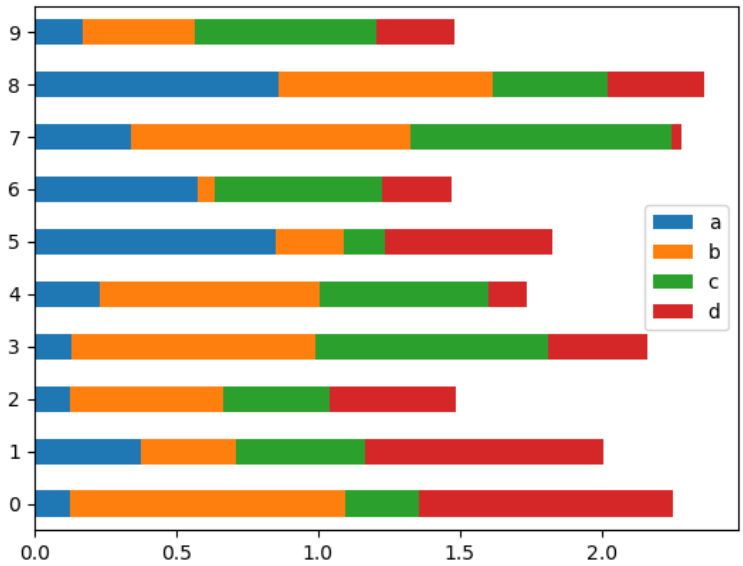



Figure 17: Stacked bar plot

To get horizontal bars, use the `barh` method:

```
In [23]: df2.plot.barh(stacked=True);
```

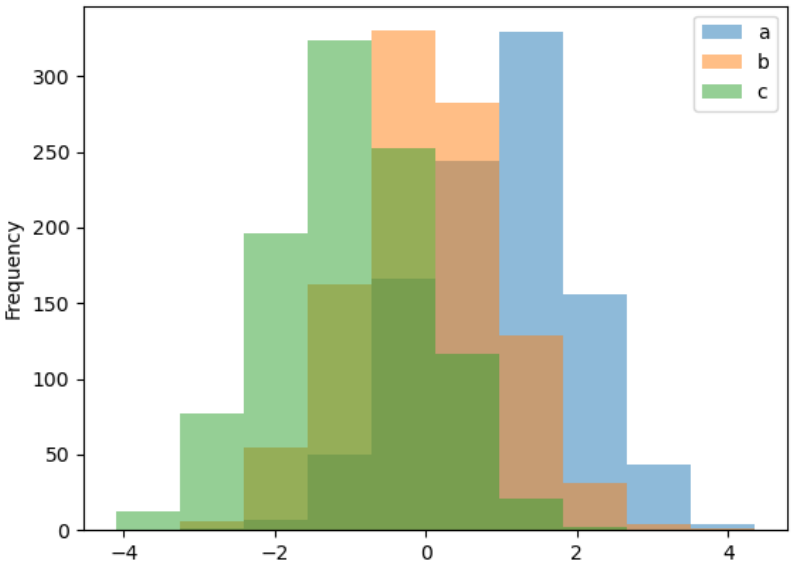



Figure 18: Horizontal stacked bar plot

# 10 Histograms

Histograms can be drawn using `DataFrame.plot.hist()` and `Series.plot.hist()`.

```
In [24]: df4 = pd.DataFrame(
    ....:     {
    ....:         "a": np.random.randn(1000) + 1,
    ....:         "b": np.random.randn(1000),
    ....:         "c": np.random.randn(1000) - 1,
    ....:     },
    ....:     columns=['a', 'b', 'c'],
    ....: )

In [25]: plt.figure();
In [26]: df4.plot.hist(alpha=0.5);
```

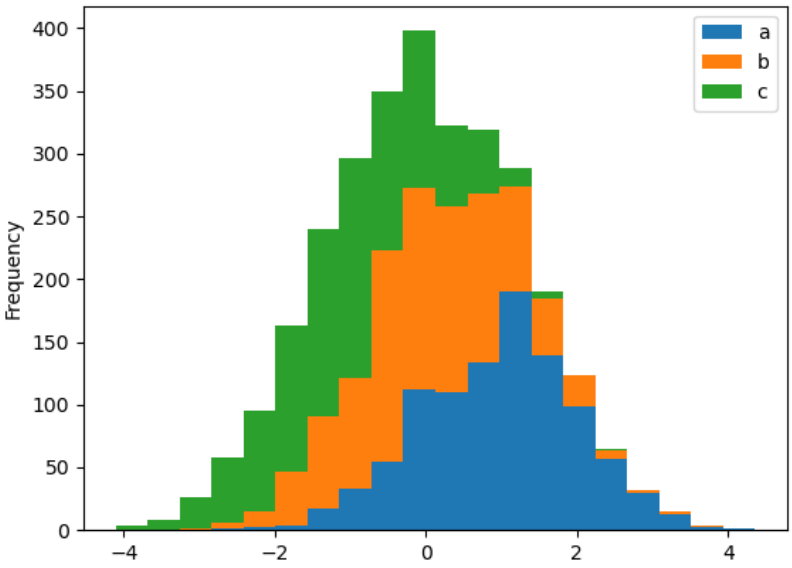



Figure 19: Overlapping histograms

You can stack a histogram using `stacked=True`. The bin size can be changed using the `bins` keyword.

```
In [27]: plt.figure();
In [28]: df4.plot.hist(stacked=True, bins=20);
```

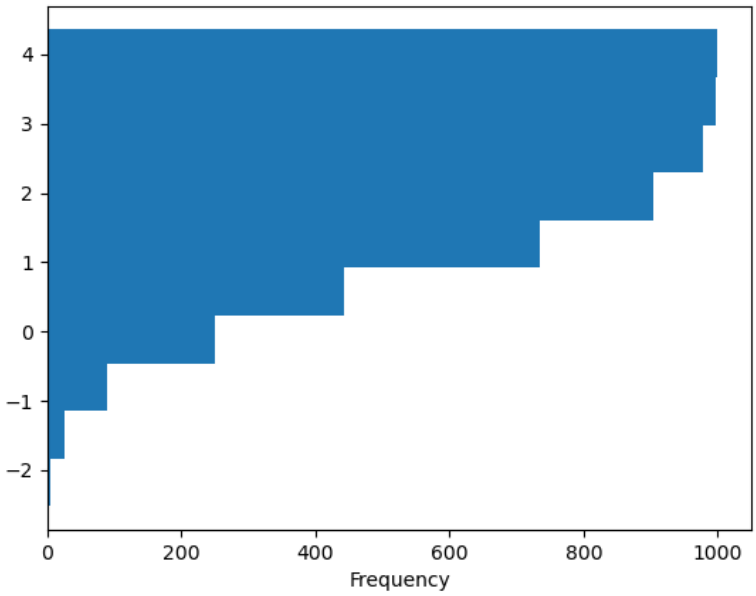



Figure 20: Stacked histogram

You can set other data supported by matplotlib hist. For example, horizontal and cumulative histograms can be drawn using `orientation='horizontal'` and `cumulative=True`.

```
In [29]: plt.figure();
In [30]: df4["a"].plot.hist(orientation="horizontal", cumulative=
    True);
```

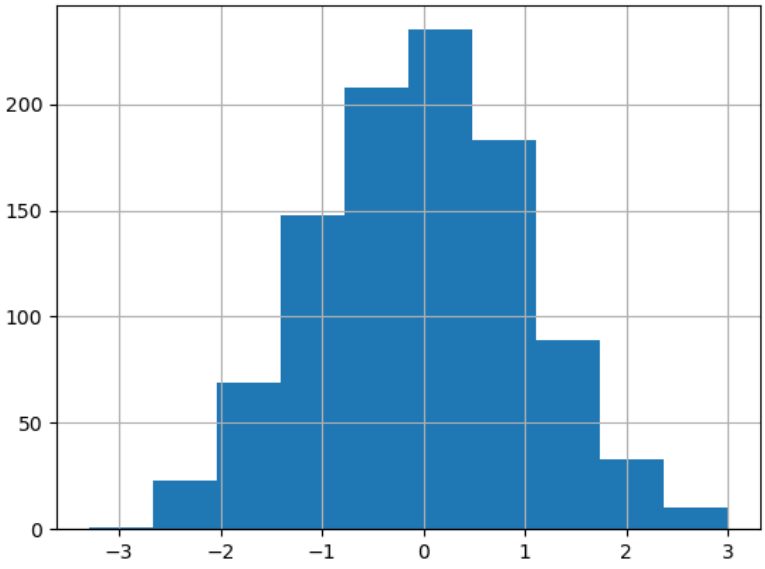



Figure 21: Horizontal cumulative histogram

For more details, see the hist method and matplotlib hist documentation. You can also use the `DataFrame.hist` interface for plotting histograms.

```
In [31]: plt.figure();
In [32]: df["A"].diff().hist();
```

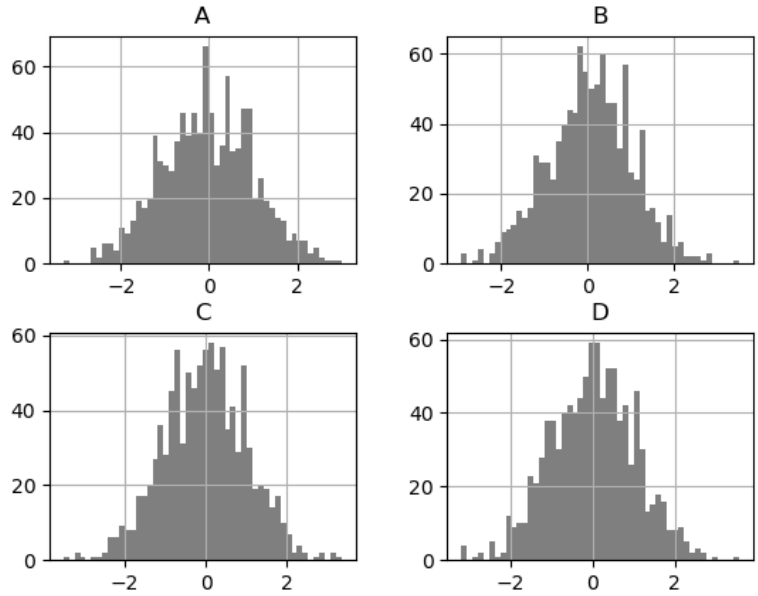



Figure 22: Histogram of differences

```
In [33]: plt.figure();
In [34]: df.diff().hist(color="k", alpha=0.5, bins=50);
```

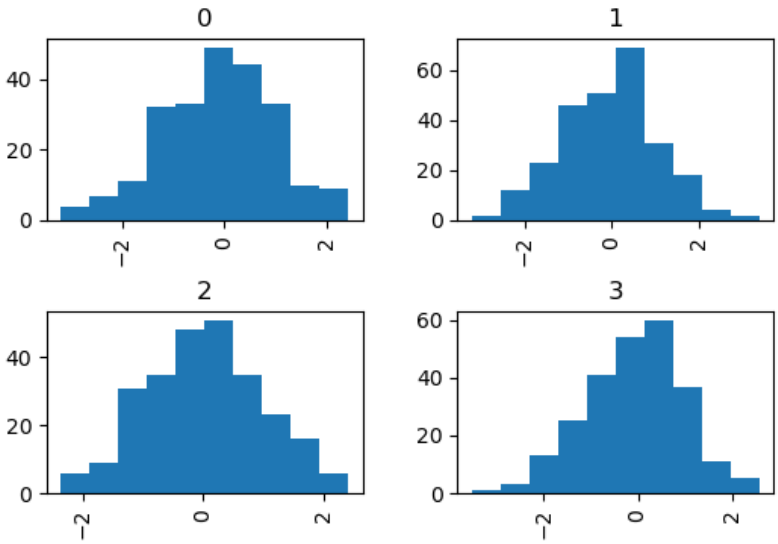



Figure 23: Histograms of differences for each column

Using `by` you can specify plotting of a group of histograms:

```
In [35]: data = pd.Series(np.random.randn(1000))
In [36]: data.hist(by=np.random.randint(0, 4, 1000), figsize=(6, 4)
   );
```

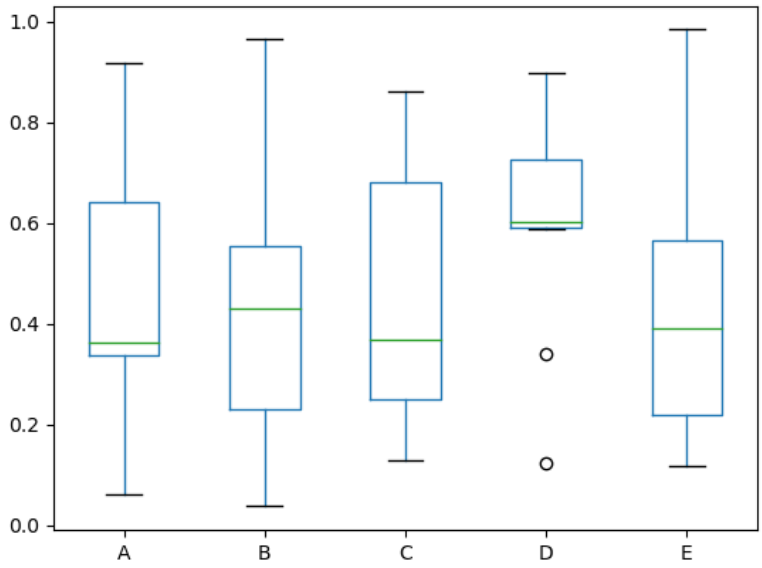



Figure 24: Grouped histograms

# 11 Box Plots

Can be created using `Series.plot.box()` and `DataFrame.plot.box()`, or `DataFrame.boxplot()` to visualize the distribution of values in each column. For example, a plot of five trials of 10 observations of a uniform random variable on $[0, 1)$.

```
In [37]: df = pd.DataFrame(np.random.rand(10, 5), columns=['A', 'B'
    , 'C', 'D', 'E'])
In [38]: df.plot.box();
```

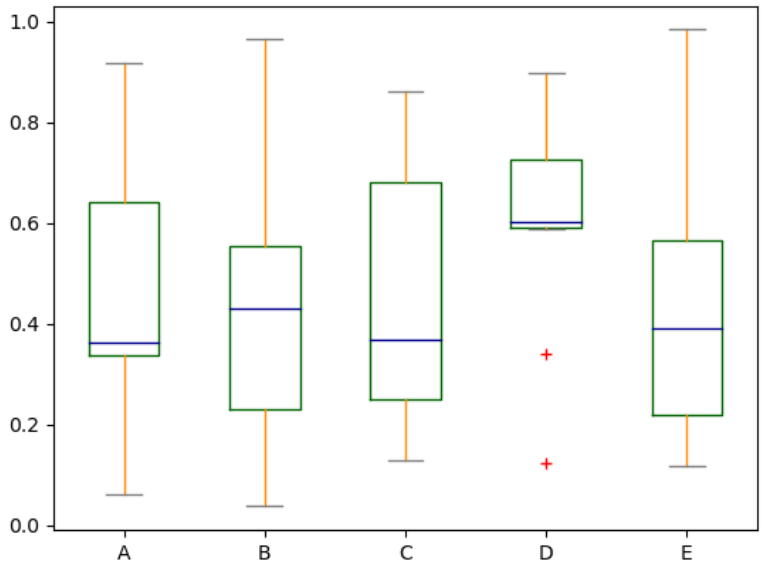



Figure 25: Box plot

In a boxplot, you can change the styling by setting `color`. You can pass a dictionary whose keys are `boxes`, `whiskers`, `medians`, and `caps`. If some keys are missing from the dictionary, default colors are used for them. In addition, boxplot has a `sym` parameter to specify the style of individual points.

```
In [39]: color = {
    ....:     "boxes": "DarkGreen",
    ....:     "whiskers": "DarkOrange",
    ....:     "medians": "DarkBlue",
    ....:     "caps": "Gray",
    ....: }

In [40]: df.plot.box(color=color, sym="r+");
```

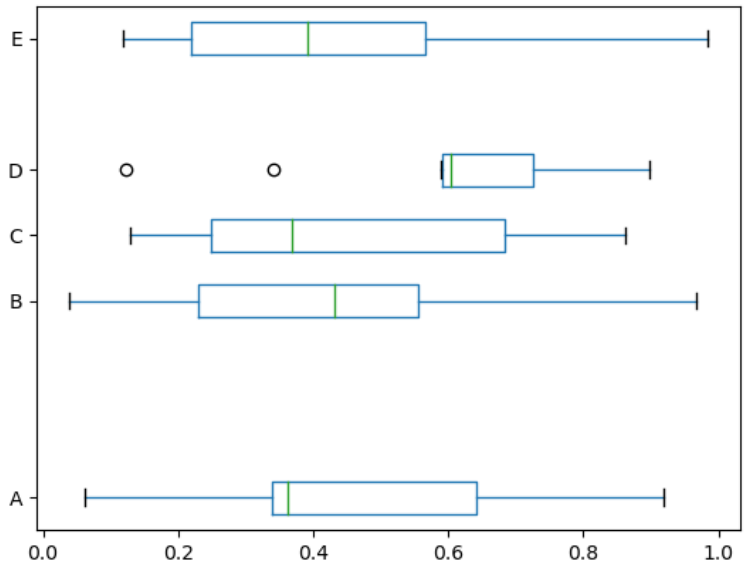



Figure 26: Styled box plot

In addition, you can pass other keywords supported by matplotlib boxplot. For example, you can draw a horizontal and arbitrarily positioned rectangular chart using `vert=False` and `positions`.

```
In [41]: df.plot.box(vert=False, positions=[1, 4, 5, 6, 8]);
```

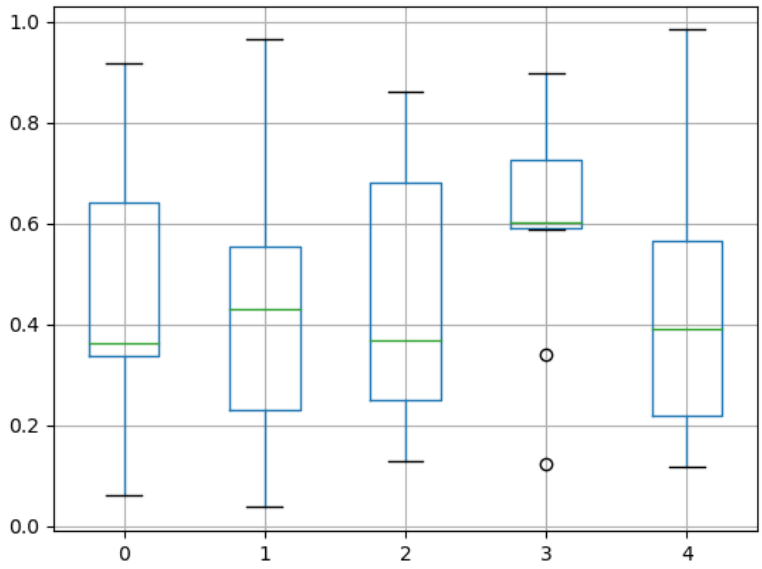



Figure 27: Horizontal box plot with custom positions

For more details, see the boxplot method and matplotlib boxplot documentation. You can also use `DataFrame.boxplot`.

```
In [42]: df = pd.DataFrame(np.random.rand(10, 5))
In [43]: plt.figure();
In [44]: bp = df.boxplot()
```

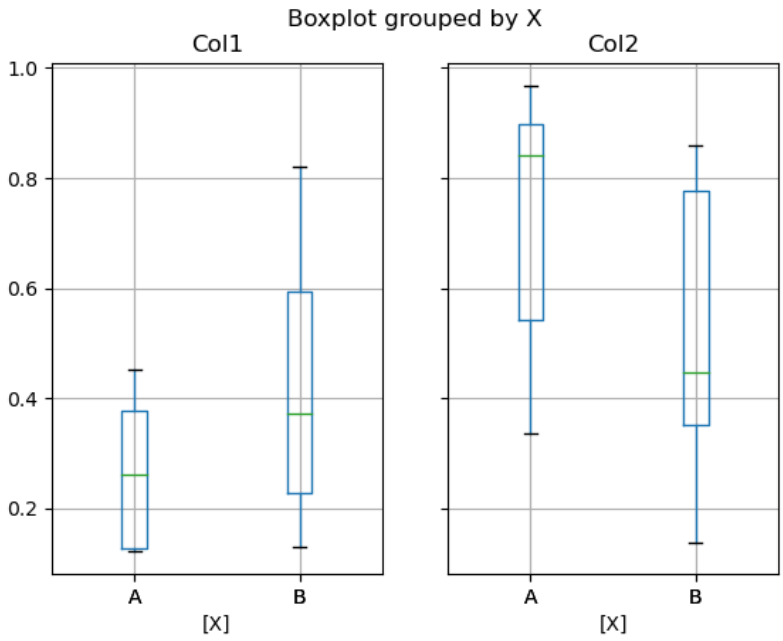



Figure 28: Box plot of DataFrame

You can create a stratified boxplot using `by` for grouping. For example,

```
In [45]: df = pd.DataFrame(np.random.rand(10, 2), columns=['Col1',
    'Col2'])
In [46]: df["x"] = pd.Series(["A", "A", "A", "A", "A", "B", "B", "B
    ", "B", "B"])
In [47]: plt.figure();
In [48]: bp = df.boxplot(by="x")
```

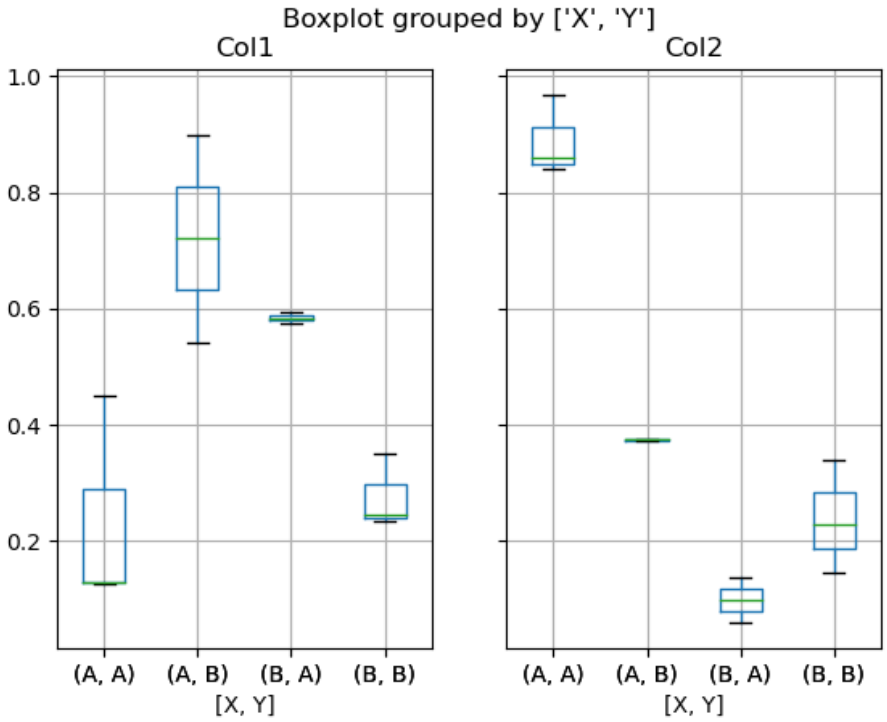



Figure 29: Box plot grouped by X

You can also specify a subset of columns to plot, as well as group columns:

```
df = pd.DataFrame(np.random.rand(10, 3), columns=['Col1', 'Col2', '
    Col3'])
df["x"] = pd.Series(['A', 'A', 'A', 'A', 'A', 'B', 'B', 'B', 'B', '
    B'])
df["y"] = pd.Series(['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', '
    B'])
plt.figure();
bp = df.boxplot(column=['Col1', 'Col2'], by=['X', 'Y'])
```

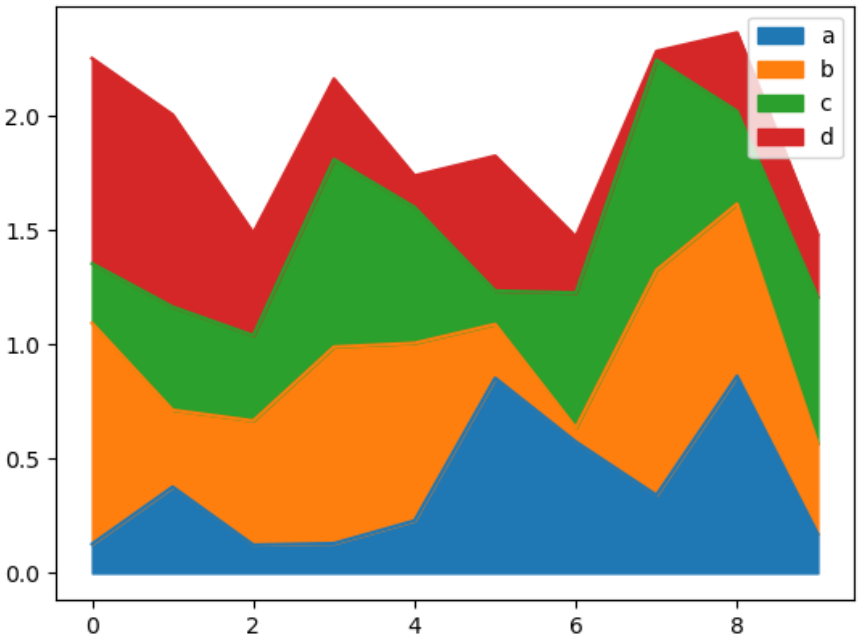



Figure 30: Box plot grouped by X and Y

# 12 Area Plot

Created by `Series.plot.area()` and `DataFrame.plot.area()`. By default, area plots are stacked. To create a stacked area plot, each column must contain either all positive or all negative values. NaN is automatically replaced with 0. If you want to drop or fill them with other values, use `dataframe.dropna()` or `dataframe.fillna()` before calling plot.

```
df = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b', 'c', 'd
    '])
df.plot.area();
```

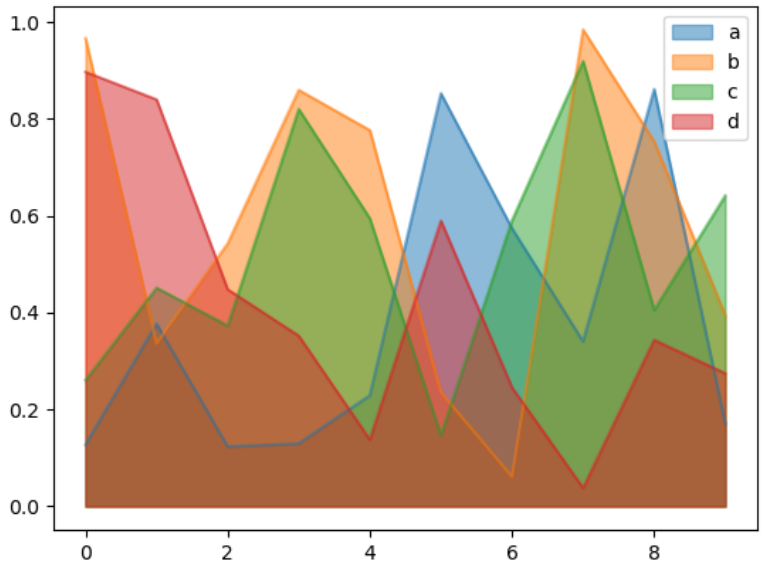



Figure 31: Stacked area plot

To create an unstacked plot, pass `stacked=False`. The transparency (alpha) is set to 0.5 unless otherwise specified:

```
In [62]: df.plot.area(stacked=False);
```

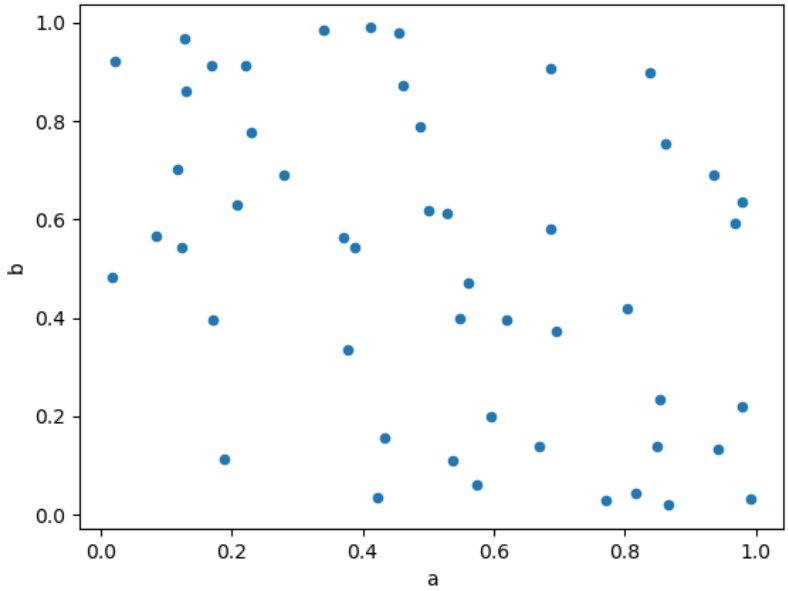



Figure 32: Unstacked area plot

# 13 Scatter Plot

Created by the `DataFrame.plot.scatter()` method. A scatter plot requires numeric columns for the x and y axes. They can be specified with the keywords `x` and `y`.

```
In [63]: df = pd.DataFrame(np.random.rand(50, 4), columns=['a', 'b'
    , 'c', 'd'])
In [64]: df.plot.scatter(x='a', y='b');
```

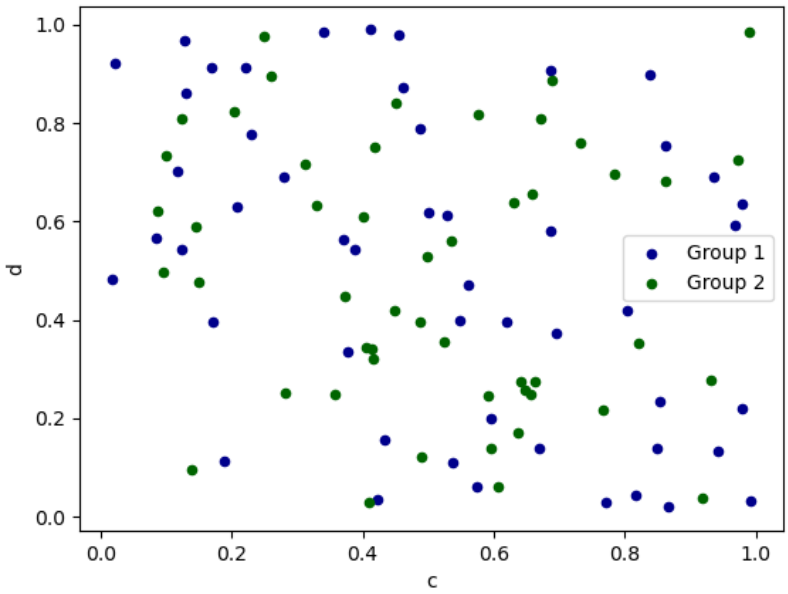



Figure 33: Scatter plot of a vs b

To plot multiple column groups in a single axes, repeat the plot method specifying the target `ax`. It is recommended to specify `color` and `label` keywords to distinguish each group.

```
ax = df.plot.scatter(x="a", y="b", color="DarkBlue", label="Group 1
    ")
df.plot.scatter(x="c", y="d", color="DarkGreen", label="Group 2",
    ax=ax);
```

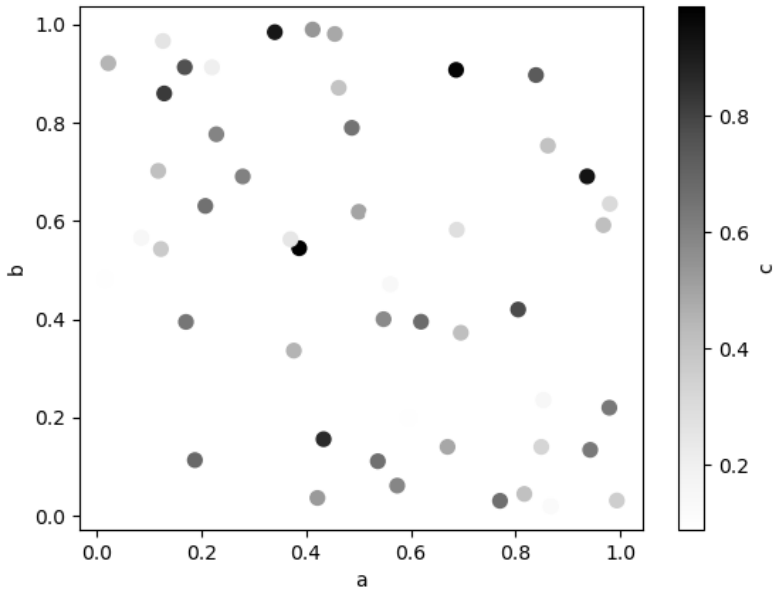



Figure 34: Two scatter groups in one axes

You can specify an additional column of colors for each point:

```
In [67]: df.plot.scatter(x="a", y="b", c="c", s=50);
```

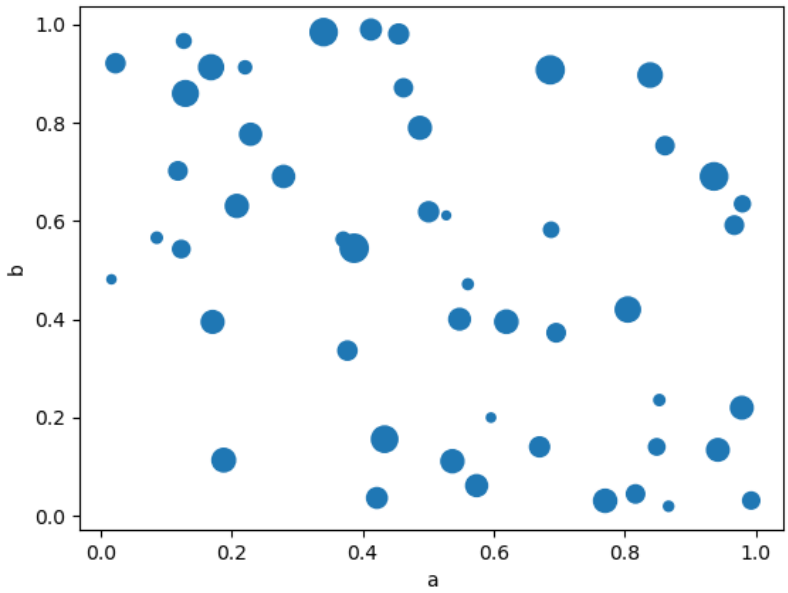



Figure 35: Scatter plot with color mapping

```
In [68]: df.plot.scatter(x="a", y="b", s="c");
```

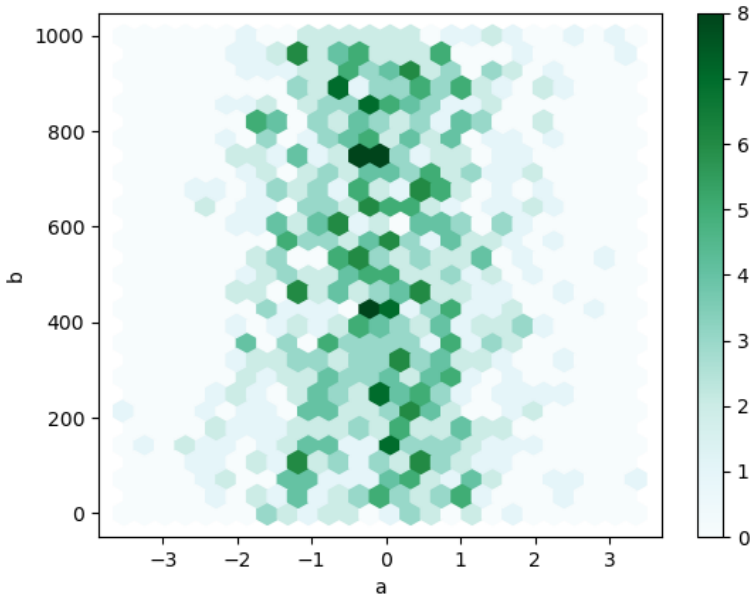



Figure 36: Scatter plot with size mapping

For more details, see the scatter method and matplotlib scatter documentation.

# 14 Hexagonal Bin Plot

Created by the `DataFrame.plot.hexbin()` method. Hexbin plots can be an alternative to scatter plots when the data is too dense to plot each point individually.

```
In [69]: df = pd.DataFrame(np.random.randn(1000, 2), columns=['a',
    'b'])
In [70]: df["b"] = df["b"] + np.arange(1000)
In [71]: df.plot.hexbin(x="a", y="b", gridsize=25);
```

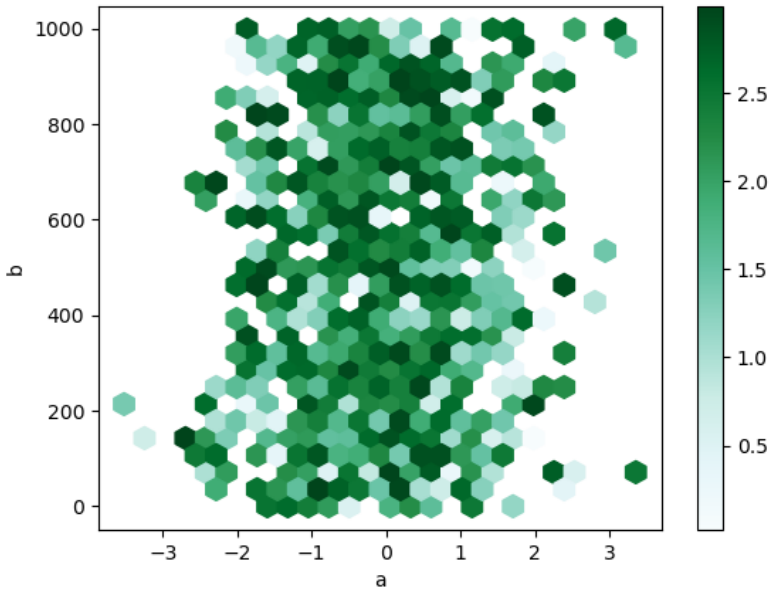



Figure 37: Hexbin plot

The `gridsize` parameter specifies the number of hexagons in the x direction and defaults to 100. By default, a histogram of values around each point $(x, y)$ is computed. You can specify alternative aggregations by passing values to the `C` and `reduce_C_function` arguments. `C` specifies the value at each point $(x, y)$, and `reduce_C_function` is a function of one argument that reduces all values in a group to a single number (e.g., mean, max, sum, standard deviation). In this example, positions are given by columns a and b, and the value is given by column z. Groups are aggregated using the `max` function from the NumPy library.

```
In [72]: df = pd.DataFrame(np.random.randn(1000, 2), columns=["a",
    "b"])
In [73]: df["b"] = df["b"] + np.arange(1000)
In [74]: df["z"] = np.random.uniform(0, 3, 1000)
In [75]: df.plot.hexbin(x="a", y="b", C="z", reduce_C_function=np.
    max, gridsize=25);
```

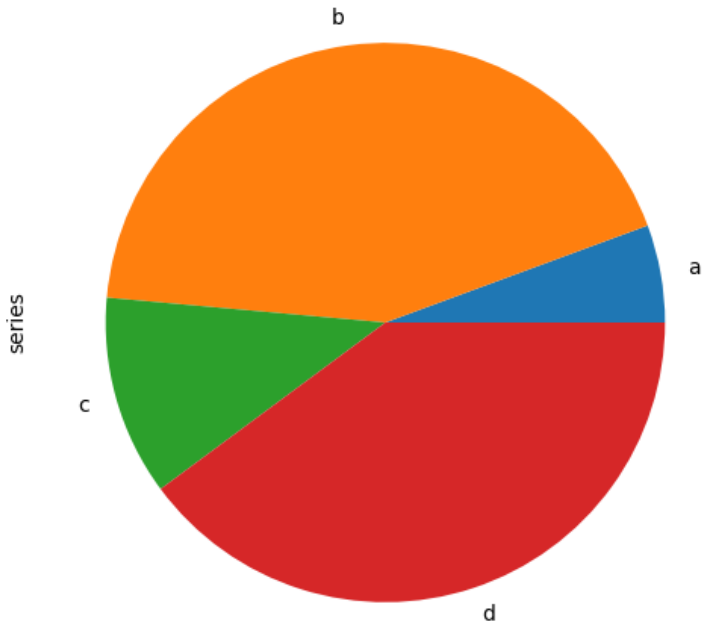



Figure 38: Hexbin plot with aggregation

See the hexbin method and matplotlib hexbin documentation.

# 15 Pie Plot

Created by the `DataFrame.plot.pie()` or `Series.plot.pie()` method. NaN is automatically replaced with 0. A `ValueError` exception will be raised if there are negative numbers among the values.

```
In [76]: series = pd.Series(3 * np.random.rand(4), index=["a", "b",
    "c", "d"], name="series")
In [77]: series.plot.pie(figsize=(6, 6));
```

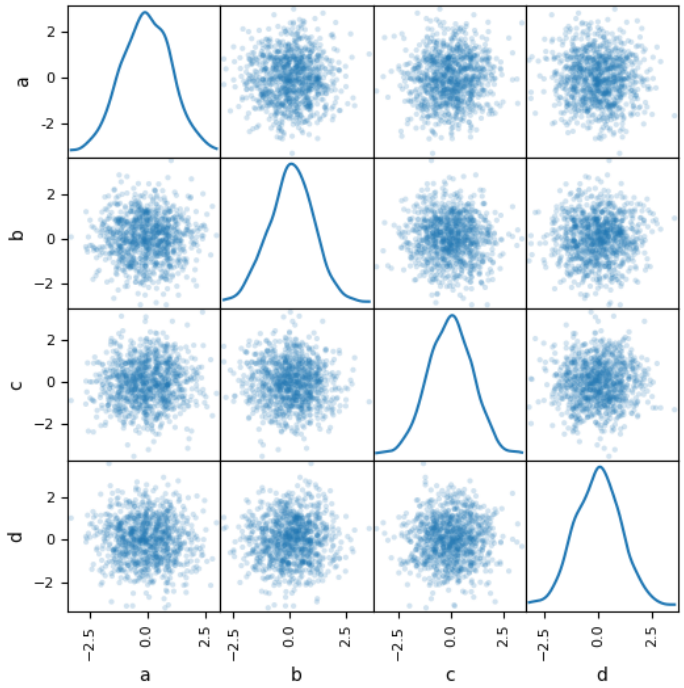



Figure 39: Pie chart

# 16 Plotting with Missing Data

The pandas package tries to be pragmatic when outputting a DataFrame or Series with missing data. Missing values are dropped, skipped, or filled depending on the plot type.

| **Plot Type** | **NaN Handling** |
|---|---|
| Line | Leave gaps at NaNs |
| Line (stacked) | Fill 0's |
| Bar | Fill 0's |
| Scatter | Drop NaNs |
| Histogram | Drop NaNs (column-wise) |
| Box | Drop NaNs (column-wise) |
| Area | Fill 0's |
| KDE | Drop NaNs (column-wise) |
| Hexbin | Drop NaNs |
| Pie | Fill 0's |

Table 2: NaN handling by plot type.

If the default actions do not suit you, use `fillna()` or `dropna()` to change the data before output.

# 17 Plotting Tools

These functions can be imported from `pandas.plotting` and take a Series or DataFrame as an argument.

## 17.1 Scatter Matrix Plot

Created by the `scatter_matrix` function from `pandas.plotting`:

```
In [83]: from pandas.plotting import scatter_matrix
In [84]: df = pd.DataFrame(np.random.randn(1000, 4), columns=['a',
    'b', 'c', 'd'])
In [85]: scatter_matrix(df, alpha=0.2, figsize=(6, 6), diagonal="
    kde");
```

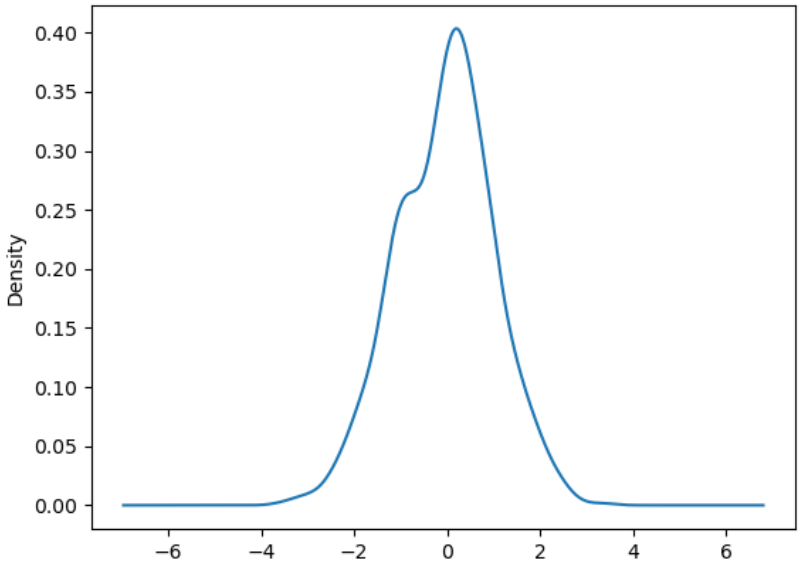



Figure 40: Scatter matrix plot

## 17.2 KDE Plot

Created by the `Series.plot.kde()` and `DataFrame.plot.kde()` methods.

```
In [86]: ser = pd.Series(np.random.randn(1000))
In [87]: ser.plot.kde();
```

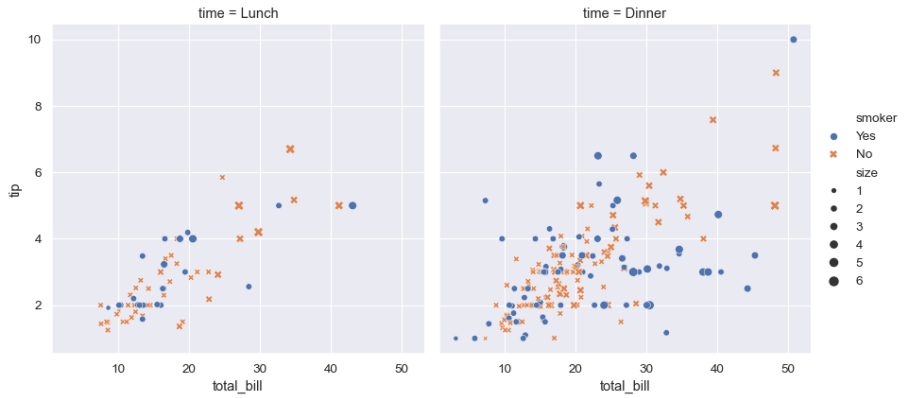



Figure 41: KDE plot

# 18 Introduction to seaborn

Seaborn is a library for creating statistical graphics in Python. It is based on matplotlib and integrates closely with pandas data structures.

Seaborn helps you explore and analyze data. Its plotting functions work with data frames and arrays containing entire datasets, and internally perform the necessary semantic mapping and statistical aggregation to create informative plots. Its declarative, dataset-oriented API allows you to focus on what the different elements of your plots mean, rather than on the details of how to draw them.

Here is an example of what seaborn can do:

```
# Import seaborn
import seaborn as sns

# Apply the default theme
sns.set_theme()

# Load an example dataset
tips = sns.load_dataset("tips")

# Create a visualization
sns.relplot(
    data=tips,
    x="total_bill", y="tip", col="time",
    hue="smoker", style="smoker", size="size",
)
```

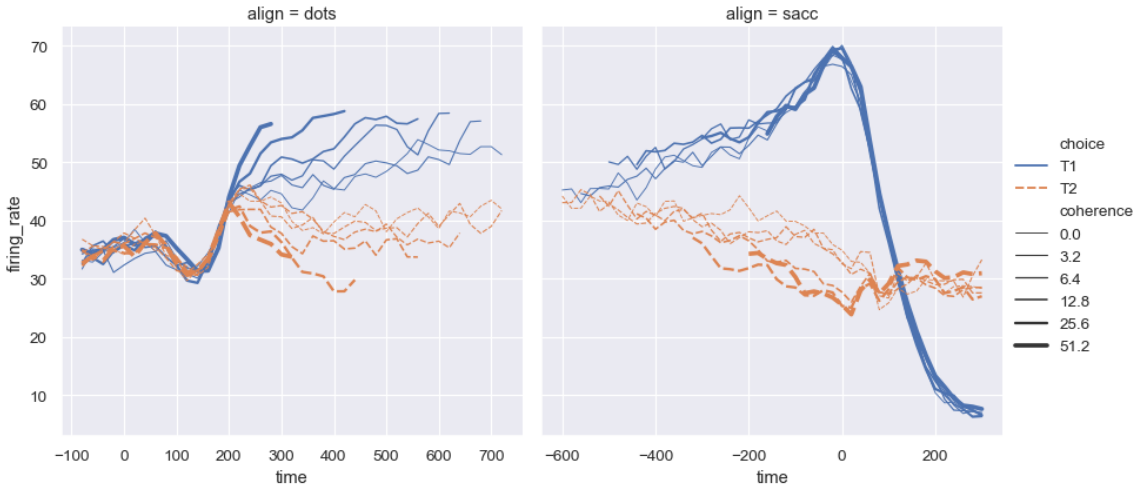



Figure 42: Seaborn relplot example

Several things happened here. Let's go through them one by one:

```
# Import seaborn
import seaborn as sns
```

Seaborn is the only library we need to import for this simple example. By convention, it is imported with the abbreviation `sns`.

Behind the scenes, seaborn uses matplotlib to draw its plots. For interactive work, it is recommended to use the Jupyter/IPython interface in matplotlib mode, otherwise you will have to call `matplotlib.pyplot.show()` when you want to see the plot.

```
# Apply the default theme
sns.set_theme()
```

This uses the matplotlib rcParam system and will affect how all matplotlib plots look, even if you don't create them with seaborn. Besides the default theme, there are several other options, and you can control the style and scaling yourself to quickly translate your work between presentation contexts (for example, create a version of your figure that will have readable fonts when projected during a talk). If you like the default matplotlib settings or prefer a different theme, you can skip this step and still use seaborn plotting functions.

```
# Load an example dataset
tips = sns.load_dataset("tips")
```

Most of the code in the documentation will use this function for quick access to example datasets. There is nothing special about these datasets: they are just pandas data frames, and we could load them with `pandas.read_csv()` or create them manually. In most examples in the documentation, data will be specified using pandas data frames, but seaborn is very flexible about the data structures it accepts.

```
# Create a visualization
sns.relplot(
    data=tips,
    x="total_bill", y="tip", col="time",
    hue="smoker", style="smoker", size="size",
)
```

This plot shows the relationship between five variables in the tips dataset using a single call to the seaborn `relplot()` function. Note that we only specified the names of the variables and their roles in the plot. Unlike using matplotlib directly, there was no need to specify plot element attributes in terms of color values or marker codes. Behind the scenes, seaborn handled the translation from values in the dataset to arguments that matplotlib understands. This declarative approach lets you focus on the questions you want to answer, rather than on the details of controlling matplotlib.

## 18.1 High-level API for Statistical Graphics

There is no universally optimal way to visualize data. Different questions are best answered by different plots. Seaborn makes it easy to switch between different visual representations with a consistent, dataset-oriented API.

The function is named this way because it is designed to visualize many different statistical relationships. While scatter plots are often effective, relationships in which one variable represents a measure of time are better represented by a line. The function has a convenient parameter that makes it easy to switch to this alternative representation: `kind`

```
dots = sns.load_dataset("dots")
sns.relplot(
    data=dots, kind="line",
    x="time", y="firing_rate", col="align",
    hue="choice", size="coherence", style="choice",
    facet_kws=dict(sharex=False),
)
```

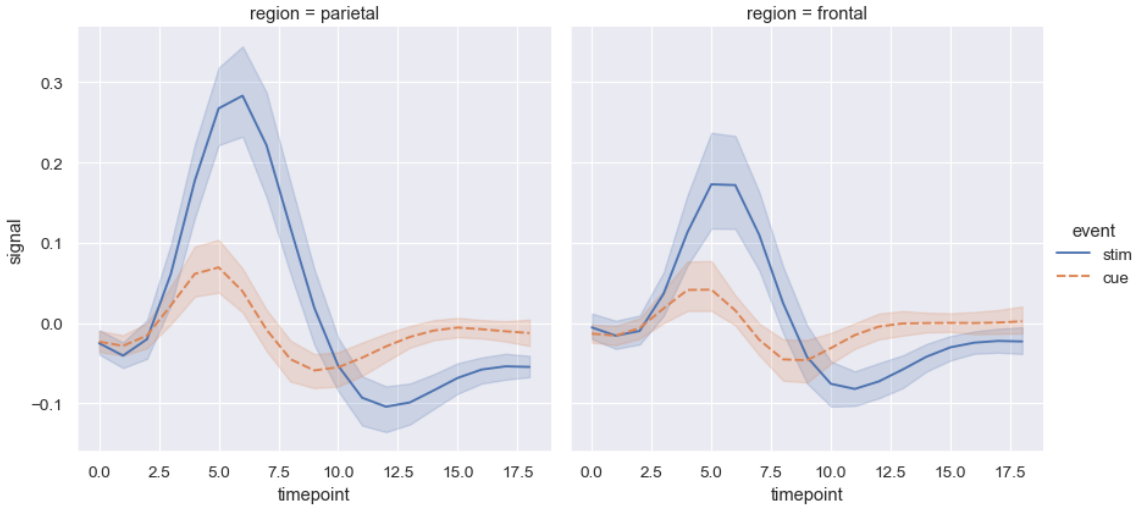



Figure 43: Seaborn line plot with facets

Notice how the `size` and `style` parameters are used in both the scatter and line plots, but they affect the two visualizations differently: changing the marker area and symbol in the scatter plot versus the line width and dash style in the line plot. We did not need to keep these details in mind, which allowed us to focus on the overall structure of the plot and the information we want to convey.

## 18.2 Statistical Estimation

Often we are interested in the average value of one variable as a function of other variables. Many seaborn functions automatically perform the statistical estimation needed to answer the following questions:

```
fmri = sns.load_dataset("fmri")
sns.relplot(
    data=fmri, kind="line",
    x="timepoint", y="signal", col="region",
    hue="event", style="event",
)
```

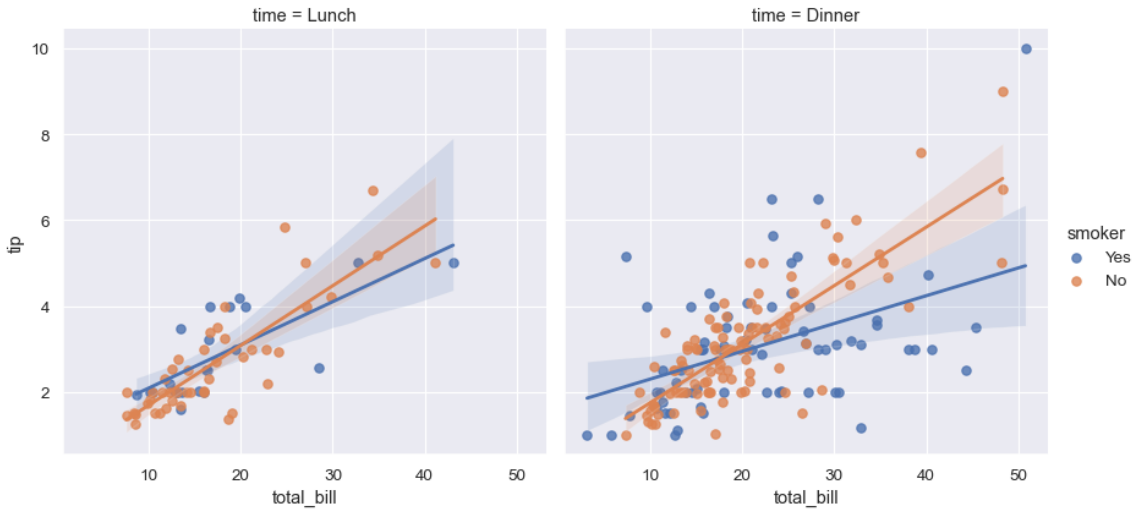



Figure 44: Seaborn line plot with confidence intervals

## 18.3 Distributional Representations

Statistical analysis requires knowledge of the distribution of variables in a dataset. The seaborn function supports several approaches to visualizing distributions. These include classical methods such as histograms and resource-intensive approaches such as kernel density estimation:

```
sns.displot(data=tips, x="total_bill", col="time", kde=True)
```

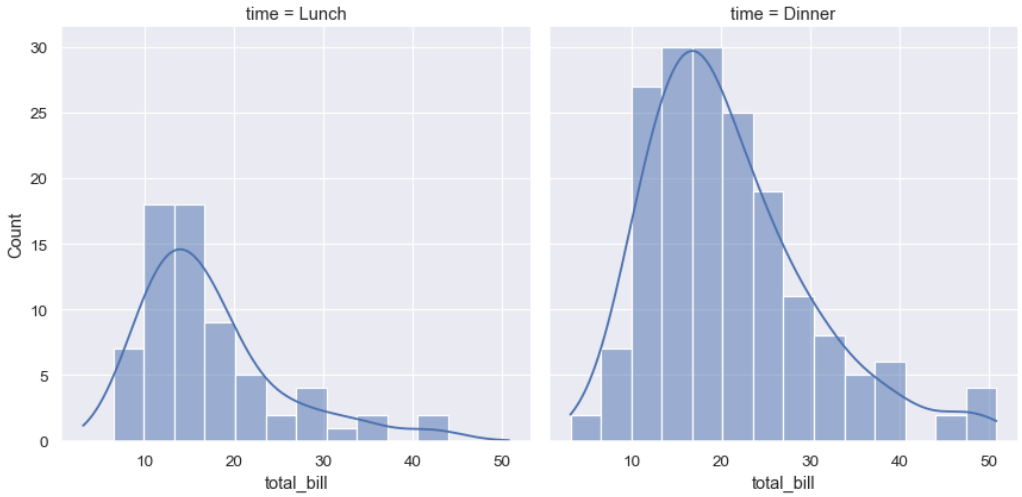



Figure 45: Seaborn histogram with KDE

Seaborn also tries to promote effective but less familiar methods, such as computing and plotting the empirical cumulative distribution function of the data:

```
sns.displot(data=tips, kind="ecdf", x="total_bill", col="time", hue
    ="smoker", rug=True)
```

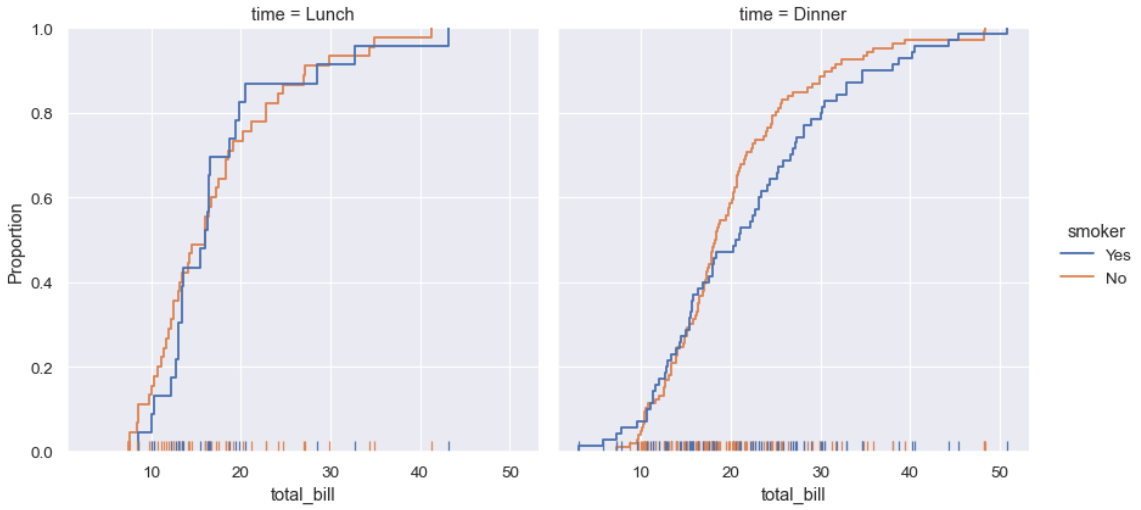



Figure 46: Seaborn ECDF plot

## 18.4 Plots for Categorical Data

Several specialized plot types in seaborn are aimed at visualizing categorical data. They can be accessed via `catplot`. These plots offer different levels of granularity. At the highest level, you may want to see each observation, drawing a "swarm" plot: a scatter plot that adjusts the position of points along the categorical axis so that they do not overlap:

```
sns.catplot(data=tips, kind="swarm", x="day", y="total_bill", hue="
    smoker")
```

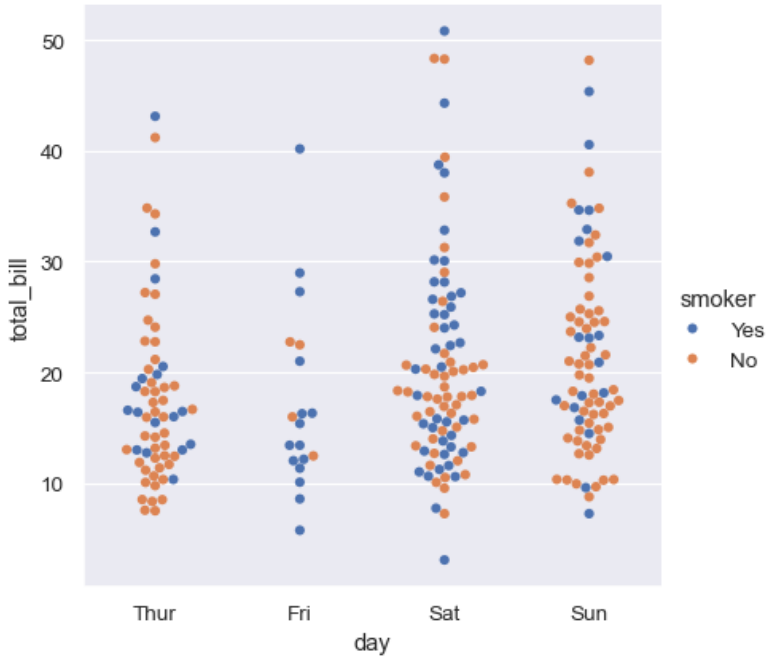



Figure 47: Seaborn swarm plot

Alternatively, you can use kernel density estimation to represent the underlying distribution from which the points are sampled:

```
sns.catplot(data=tips, kind="violin", x="day", y="total_bill", hue=
    "smoker", split=True)
```

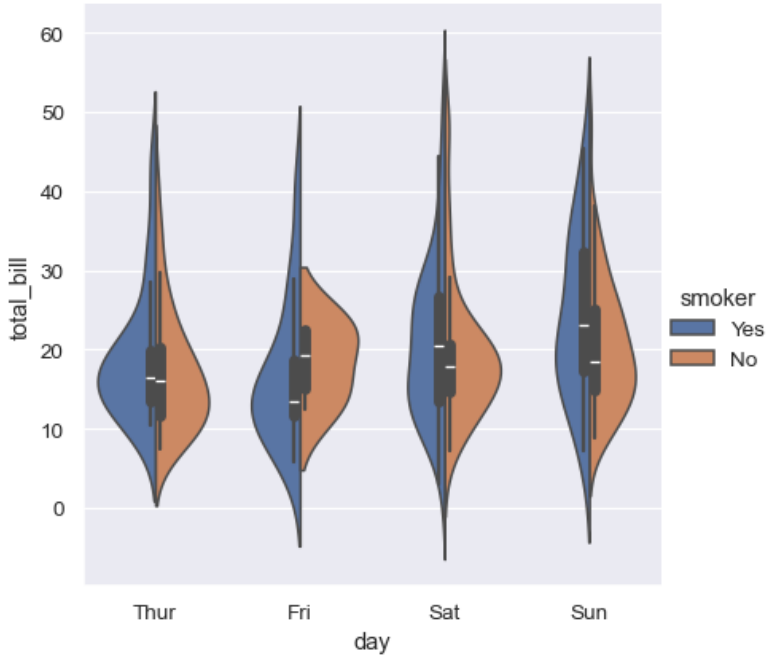



Figure 48: Seaborn violin plot

Or you can show only the mean and its confidence interval within each nested category:

```
sns.catplot(data=tips, kind="bar", x="day", y="total_bill", hue="
    smoker")
```

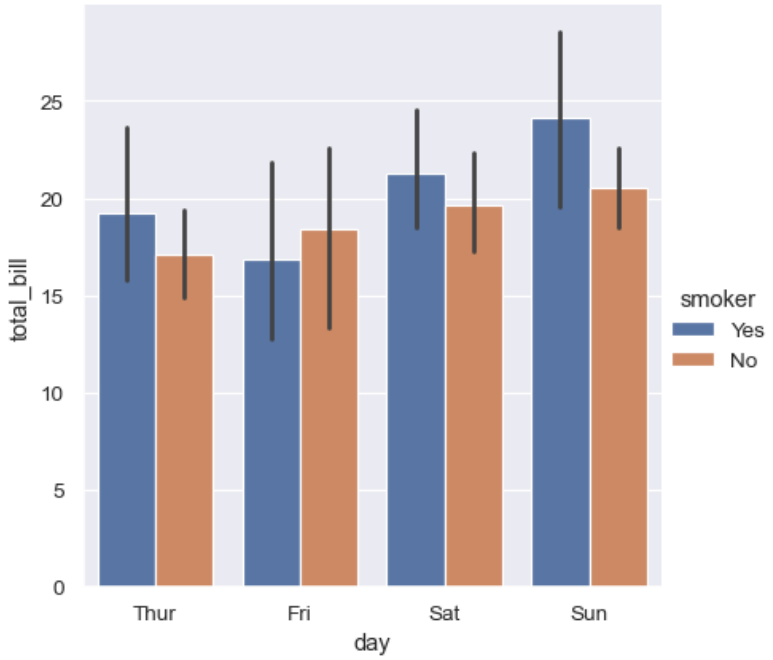



Figure 49: Seaborn bar plot with confidence intervals

## 18.5 Multivariate Representations for Complex Datasets

Some seaborn functions combine multiple plot types to quickly obtain informative summaries of a dataset. The first, `jointplot`, focuses on a single relationship. It plots the joint distribution between two variables along with the marginal distribution of each variable:

```
penguins = sns.load_dataset("penguins")
sns.jointplot(data=penguins, x="flipper_length_mm", y="
    bill_length_mm", hue="species")
```

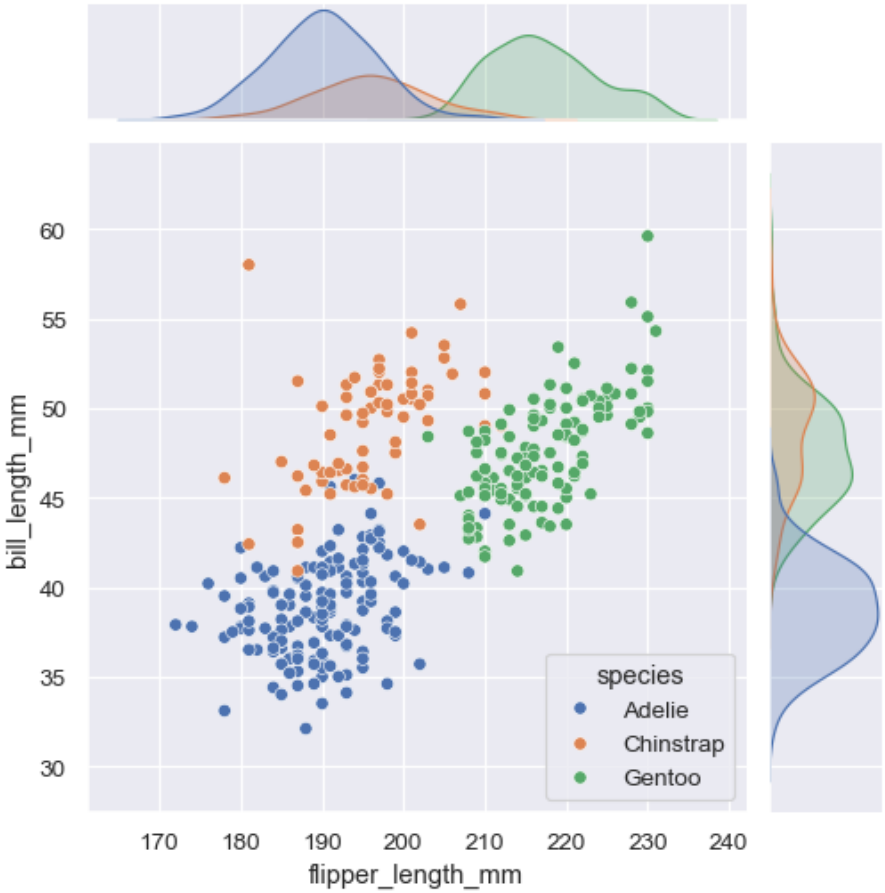



Figure 50: Seaborn jointplot

The second, `pairplot`, takes a broader view: it shows the joint and marginal distributions for all pairs of variables in a dataset:

```
sns.pairplot(data=penguins, hue="species")
```

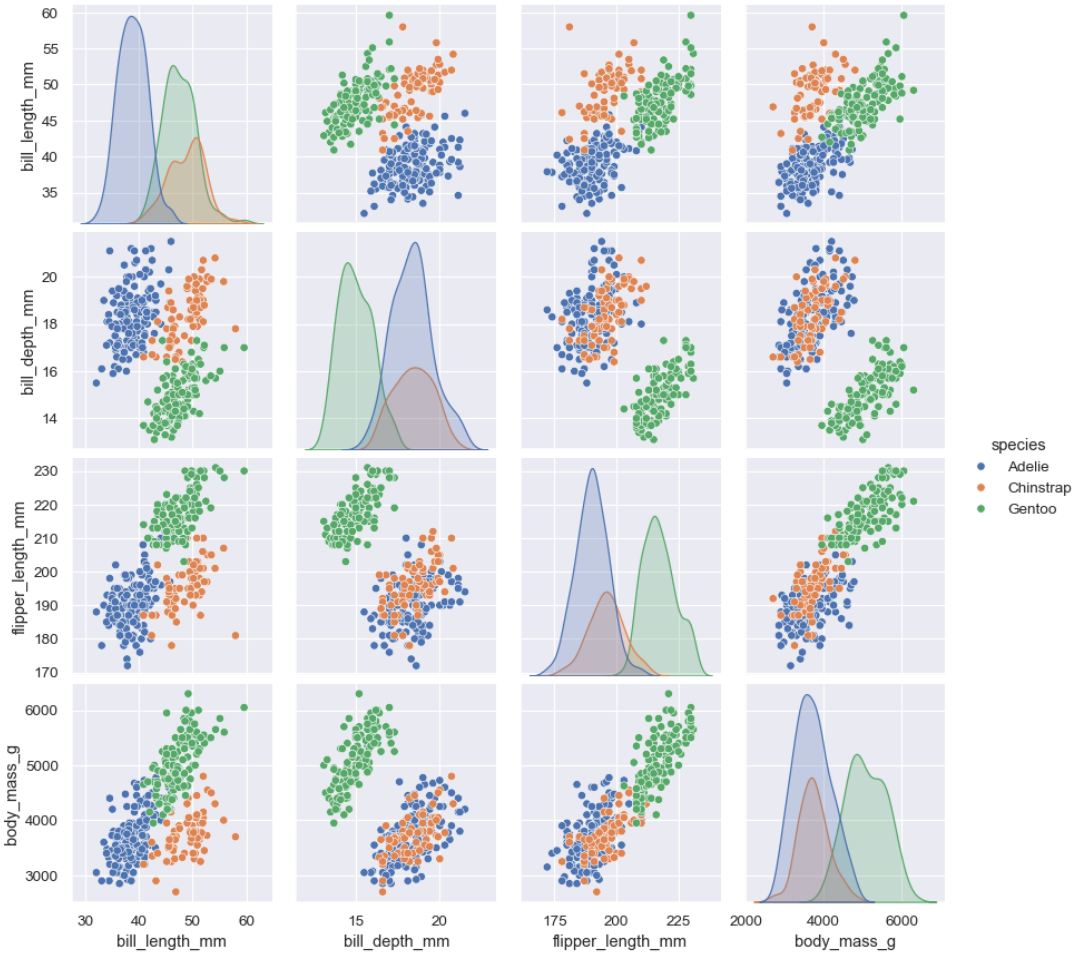



Figure 51: Seaborn pairplot

Support for faceting is built directly into these functions, but they also return a `PairGrid` object that can be used to create more complex figures. For example, you can plot a different plot type on the diagonal and add a legend:

```
g = sns.PairGrid(penguins, hue="species", corner=True)
g.map_lower(sns.kdeplot, hue=None, levels=5, color=".2")
g.map_lower(sns.scatterplot, marker="+")
g.map_diag(sns.histplot, element="step", linewidth=0, kde=True)
g.add_legend(frameon=True)
g.legend.set_bbox_to_anchor((.61, .6))
```

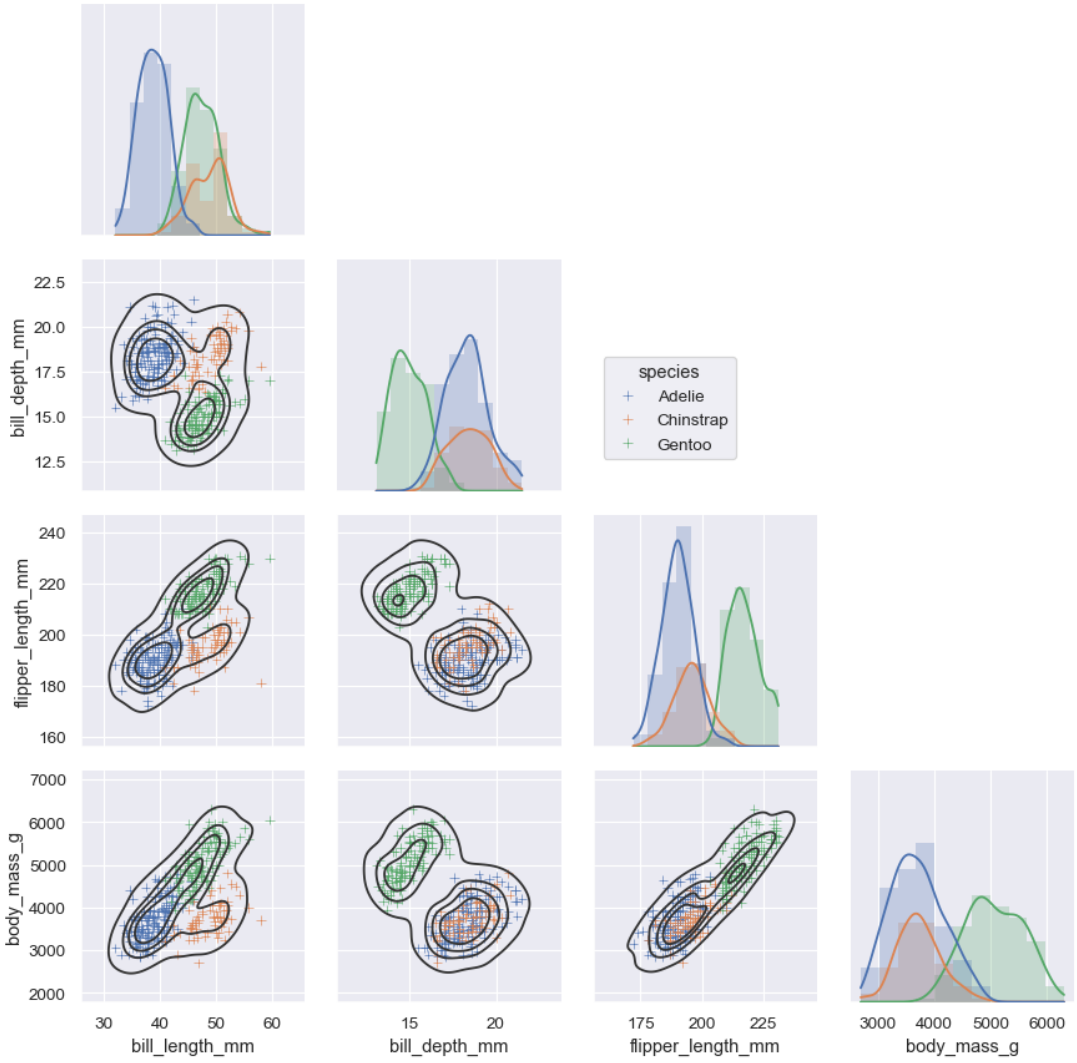



Figure 52: Seaborn PairGrid

## 18.6 Confident Defaults and Flexible Customization

Seaborn creates complete plots with a single function call: when possible, its functions automatically add informative axis labels and legends explaining the semantic mappings in the plot.

In many cases, seaborn also chooses defaults for its parameters based on the characteristics of the data. For example, the color mappings we have seen so far used different hues (blue, orange, and sometimes green) to represent different levels of the categorical variables assigned to `hue`. When mapping a numeric variable, some functions will switch to a continuous gradient:

```
sns.relplot(
    data=penguins,
    x="bill_length_mm", y="bill_depth_mm", hue="body_mass_g"
)
```

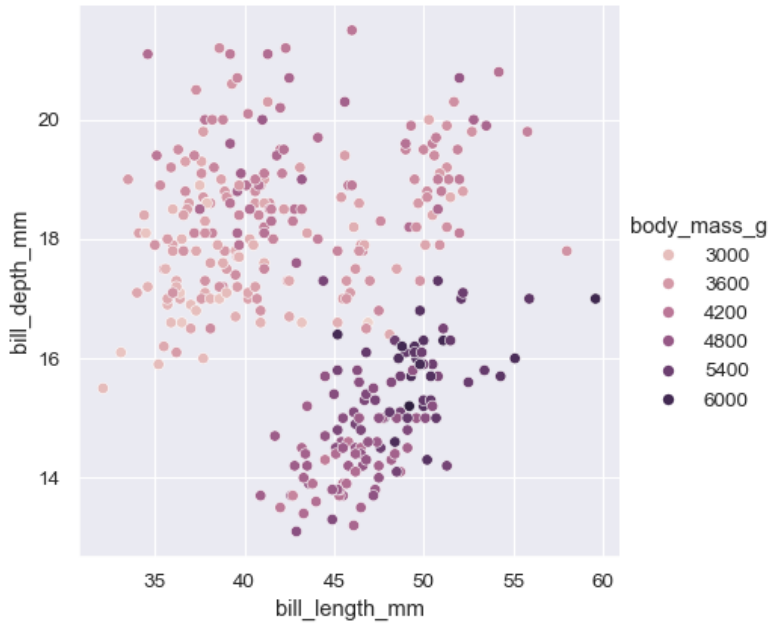



Figure 53: Seaborn continuous color mapping

When you are ready to share or publish your work, you will probably want to polish the figure beyond the default. Seaborn allows several levels of customization. It defines several built-in themes that apply to all figures, its functions have standardized parameters that can modify the semantic mappings for each plot, and additional keyword arguments are passed down to the underlying matplotlib artists, providing even more control. Once you have created a plot, its properties can be modified both through the seaborn API and by dropping down to the matplotlib layer for fine-tuning:

```
sns.set_theme(style="ticks", font_scale=1.25)
g = sns.relplot(
    data=penguins,
    x="bill_length_mm", y="bill_depth_mm", hue="body_mass_g",
    palette="crest", marker="x", s=100,
)
g.set_axis_labels("Bill length (mm)", "Bill depth (mm)", labelpad
   =10)
g.legend.set_title("Body mass (g)")
g.figure.set_size_inches(6.5, 4.5)
g.ax.margins(.15)
g.despine(trim=True)
```

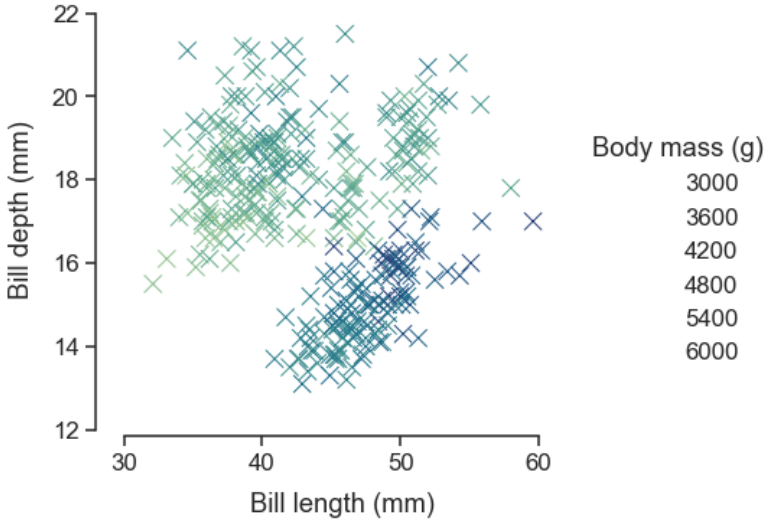



Figure 54: Customized seaborn plot

## 18.7 Relationship with matplotlib

Seaborn's integration with matplotlib allows you to use it in many of the environments that matplotlib supports, including exploratory analysis in notebooks, real-time interaction in GUI applications, and archival output in a number of raster and vector formats.

Although you can work productively using only seaborn functions, full customization of plots will require some knowledge of matplotlib concepts and API. One aspect of the learning curve for new seaborn users will be knowing when to drop down to the matplotlib layer to achieve a particular customization. On the other hand, users coming from matplotlib will find that much of their knowledge transfers.

Matplotlib has a comprehensive and powerful API; virtually any attribute of a figure can be changed to your liking. The combination of seaborn's high-level interface and matplotlib's extensive customization capabilities will allow you to quickly explore data and create plots that can be adapted to a publication-quality final product.

## 18.8 Next Steps

You have several options for where to go next. You may want to first learn how to install seaborn. Once that is done, you can browse the example gallery to get a broader sense of what plots seaborn can produce. Or you can read the rest of the user guide and tutorial for a deeper discussion of the various tools and what they are designed for. If you have a specific plot in mind and want to learn how to create it, you can consult the API reference, which describes the parameters of each function and shows many examples to illustrate usage.

# References

1. https://matplotlib.org/stable/contents.html

2. https://pandas.pydata.org/pandas-docs/stable/user_guide/visualization.html

3. https://seaborn.pydata.org/index.html

4. Plas J. Vander. *Python for Complex Tasks: Data Science and Machine Learning.* O'Reilly Best Sellers Series. St. Petersburg: Piter, 2018. 576 p.

5. Wes McKinney. *Python and Data Analysis.* Moscow: DMK Press, 2015. 482 p.

6. Abdrakhmanov M.I. *Data Visualization.* 2020. 412 p.

7. Gruzdev A. V. *Preliminary Data Preparation in Python: Volume 1. Tools and Validation.* Moscow: DMK Press, 2023. 816 p.

8. Gruzdev A. V. *Preliminary Data Preparation in Python. Volume 2: Plan, Examples, and Quality Metrics.* Moscow: DMK Press, 2023. 814 p.